

Pipeline3

March 27, 2025

1 Región Pampaneira

Debido al largo contenido del primer Pipeline, que incluye un análisis exploratorio de los datos y teniendo en cuenta que hay que restringirse a la región de Pampaneira para realizar el el objetivo principal de generar datos mediante modelos transformer, se ha creado un Notebook Pipeline2.

```
[1]: import pandas as pd
from transformers import pipeline

pd.options.mode.chained_assignment = None

pd.reset_option('display.max_rows') # To show all rows
pd.reset_option('display.max_columns') # To show all columns
```

```
[2]: trafico_feb22_ago23 = pd.read_csv('../data/trafico_feb22_ago23.csv')
```

```
[3]: # PAM_1 y BUB
columns = ['vehicles_PAM_1_OUT',
'vehicles_PAM_1_OUT_Zona_Granada',
'vehicles_PAM_1_OUT_Zona_Catalunia_y_Otras',
'vehicles_PAM_1_OUT_Zona_Andalucia_no_GR',
'vehicles_PAM_1_OUT_Extranjero',
'vehicles_PAM_1_OUT_Zona_Comunidad_de_Madrid',
'vehicles_PAM_1_OUT_Zona_Extremadura_y_Otras',
'vehicles_PAM_1_OUT_Zona_Otras',
'vehicles_PAM_1_OUT_5_seats',
'vehicles_PAM_1_OUT_+5_seats',
'vehicles_PAM_1_OUT_-5_seats',
'vehicles_PAM_1_OUT_nan_seats',
'vehicles_PAM_1_OUT_101-200_CO2',
'vehicles_PAM_1_OUT_0-100_CO2',
'vehicles_PAM_1_OUT_nan_CO2',
'vehicles_PAM_1_OUT_201-300_CO2',
'vehicles_PAM_1_OUT_+300_CO2',
'vehicles_PAM_1_IN',
'vehicles_PAM_1_IN_Zona_Granada',
'vehicles_PAM_1_IN_Zona_Catalunia_y_Otras',
'vehicles_PAM_1_IN_Zona_Andalucia_no_GR',
```

```
'vehicles_PAM_1_IN_Extranjero',
'vehicles_PAM_1_IN_Zona_Comunidad_de_Madrid',
'vehicles_PAM_1_IN_Zona_Extremadura_y_Otras',
'vehicles_PAM_1_IN_Zona_Otras',
'vehicles_PAM_1_IN_5_seats',
'vehicles_PAM_1_IN_+5_seats',
'vehicles_PAM_1_IN_-5_seats',
'vehicles_PAM_1_IN_nan_seats',
'vehicles_PAM_1_IN_101-200_CO2',
'vehicles_PAM_1_IN_0-100_CO2',
'vehicles_PAM_1_IN_nan_CO2',
'vehicles_PAM_1_IN_201-300_CO2',
'vehicles_PAM_1_IN_+300_CO2',
'vehicles_PAM_2_OUT',
'vehicles_PAM_2_OUT_Zona_Granada',
'vehicles_PAM_2_OUT_Zona_Catalunia_y_Otras',
'vehicles_PAM_2_OUT_Zona_Andalucia_no_GR',
'vehicles_PAM_2_OUT_Extranjero',
'vehicles_PAM_2_OUT_Zona_Comunidad_de_Madrid',
'vehicles_PAM_2_OUT_Zona_Extremadura_y_Otras',
'vehicles_PAM_2_OUT_Zona_Otras',
'vehicles_PAM_2_OUT_5_seats',
'vehicles_PAM_2_OUT_+5_seats',
'vehicles_PAM_2_OUT_-5_seats',
'vehicles_PAM_2_OUT_nan_seats',
'vehicles_PAM_2_OUT_101-200_CO2',
'vehicles_PAM_2_OUT_0-100_CO2',
'vehicles_PAM_2_OUT_nan_CO2',
'vehicles_PAM_2_OUT_201-300_CO2',
'vehicles_PAM_2_OUT_+300_CO2',
'vehicles_PAM_2_IN',
'vehicles_PAM_2_IN_Zona_Granada',
'vehicles_PAM_2_IN_Zona_Catalunia_y_Otras',
'vehicles_PAM_2_IN_Zona_Andalucia_no_GR',
'vehicles_PAM_2_IN_Extranjero',
'vehicles_PAM_2_IN_Zona_Comunidad_de_Madrid',
'vehicles_PAM_2_IN_Zona_Extremadura_y_Otras',
'vehicles_PAM_2_IN_Zona_Otras',
'vehicles_PAM_2_IN_5_seats',
'vehicles_PAM_2_IN_+5_seats',
'vehicles_PAM_2_IN_-5_seats',
'vehicles_PAM_2_IN_nan_seats',
'vehicles_PAM_2_IN_101-200_CO2',
'vehicles_PAM_2_IN_0-100_CO2',
'vehicles_PAM_2_IN_nan_CO2',
'vehicles_PAM_2_IN_201-300_CO2',
'vehicles_PAM_2_IN_+300_CO2',
```

```

'vehicles_BUB_OUT',
'vehicles_BUB_OUT_Zona_Granada',
'vehicles_BUB_OUT_Zona_Catalunia_y_Otras',
'vehicles_BUB_OUT_Zona_Andalucia_no_GR',
'vehicles_BUB_OUT_Extranjero',
'vehicles_BUB_OUT_Zona_Comunidad_de_Madrid',
'vehicles_BUB_OUT_Zona_Extremadura_y_Otras',
'vehicles_BUB_OUT_Zona_Otras',
'vehicles_BUB_OUT_5_seats',
'vehicles_BUB_OUT_+5_seats',
'vehicles_BUB_OUT_-5_seats',
'vehicles_BUB_OUT_nan_seats',
'vehicles_BUB_OUT_101-200_CO2',
'vehicles_BUB_OUT_0-100_CO2',
'vehicles_BUB_OUT_nan_CO2',
'vehicles_BUB_OUT_201-300_CO2',
'vehicles_BUB_OUT_+300_CO2',
'vehicles_BUB_IN',
'vehicles_BUB_IN_Zona_Granada',
'vehicles_BUB_IN_Zona_Catalunia_y_Otras',
'vehicles_BUB_IN_Zona_Andalucia_no_GR',
'vehicles_BUB_IN_Extranjero',
'vehicles_BUB_IN_Zona_Comunidad_de_Madrid',
'vehicles_BUB_IN_Zona_Extremadura_y_Otras',
'vehicles_BUB_IN_Zona_Otras',
'vehicles_BUB_IN_5_seats',
'vehicles_BUB_IN_+5_seats',
'vehicles_BUB_IN_-5_seats',
'vehicles_BUB_IN_nan_seats',
'vehicles_BUB_IN_101-200_CO2',
'vehicles_BUB_IN_0-100_CO2',
'vehicles_BUB_IN_nan_CO2',
'vehicles_BUB_IN_201-300_CO2',
'vehicles_BUB_IN_+300_CO2']

trafico_feb22_ago23["date"] = pd.to_datetime(trafico_feb22_ago23["date"])
trafico_feb22_ago23["date"] = trafico_feb22_ago23["date"].dt.tz_convert('UTC')

df_orig = trafico_feb22_ago23[["date"] + columns]
columnas_int64 = df_orig.select_dtypes(include='int64').columns

# Convertir esas columnas a float64
df_orig[columnas_int64] = df_orig[columnas_int64].astype('float64')

df_orig

```

```
[3]:
      date  vehicles_PAM_1_OUT \
0    2022-02-04 12:00:00+00:00    0.0
1    2022-02-04 13:00:00+00:00    0.0
2    2022-02-04 14:00:00+00:00    0.0
3    2022-02-04 15:00:00+00:00    0.0
4    2022-02-04 16:00:00+00:00    0.0
...
10025 2023-08-31 19:00:00+00:00    18.0
10026 2023-08-31 20:00:00+00:00     1.0
10027 2023-08-31 21:00:00+00:00     0.0
10028 2023-08-31 22:00:00+00:00     0.0
10029 2023-08-31 23:00:00+00:00     0.0
```

```
      vehicles_PAM_1_OUT_Zona_Granada \
0                                0.0
1                                0.0
2                                0.0
3                                0.0
4                                0.0
...
10025                            8.0
10026                            1.0
10027                            0.0
10028                            0.0
10029                            0.0
```

```
      vehicles_PAM_1_OUT_Zona_Catalunia_y_Otras \
0                                0.0
1                                0.0
2                                0.0
3                                0.0
4                                0.0
...
10025                            5.0
10026                            0.0
10027                            0.0
10028                            0.0
10029                            0.0
```

```
      vehicles_PAM_1_OUT_Zona_Andalucia_no_GR  vehicles_PAM_1_OUT_Extranjero \
0                                0.0                                0.0
1                                0.0                                0.0
2                                0.0                                0.0
3                                0.0                                0.0
4                                0.0                                0.0
...
10025                            1.0                                1.0
```

10026	0.0	0.0
10027	0.0	0.0
10028	0.0	0.0
10029	0.0	0.0

	vehicles_PAM_1_OUT_Zona_Comunidad_de_Madrid \
0	0.0
1	0.0
2	0.0
3	0.0
4	0.0
...	...
10025	1.0
10026	0.0
10027	0.0
10028	0.0
10029	0.0

	vehicles_PAM_1_OUT_Zona_Extremadura_y_Otras \
0	0.0
1	0.0
2	0.0
3	0.0
4	0.0
...	...
10025	2.0
10026	0.0
10027	0.0
10028	0.0
10029	0.0

	vehicles_PAM_1_OUT_Zona_Otras	vehicles_PAM_1_OUT_5_seats	...	\
0	0.0	0.0	...	
1	0.0	0.0	...	
2	0.0	0.0	...	
3	0.0	0.0	...	
4	0.0	0.0	...	
...	...	...	...	
10025	0.0	16.0	...	
10026	0.0	1.0	...	
10027	0.0	0.0	...	
10028	0.0	0.0	...	
10029	0.0	0.0	...	

	vehicles_BUB_IN_Zona_Otras	vehicles_BUB_IN_5_seats	\
0	0.0	0.0	
1	0.0	0.0	

2	0.0	0.0
3	0.0	0.0
4	0.0	0.0
...	...	...
10025	0.0	28.0
10026	0.0	21.0
10027	0.0	0.0
10028	0.0	0.0
10029	0.0	0.0

	vehicles_BUB_IN_+5_seats	vehicles_BUB_IN_-5_seats \
0	0.0	0.0
1	0.0	0.0
2	0.0	0.0
3	0.0	0.0
4	0.0	0.0
...	...	...
10025	6.0	3.0
10026	2.0	1.0
10027	0.0	0.0
10028	0.0	0.0
10029	0.0	0.0

	vehicles_BUB_IN_nan_seats	vehicles_BUB_IN_101-200_CO2 \
0	0.0	0.0
1	0.0	0.0
2	0.0	0.0
3	0.0	0.0
4	0.0	0.0
...	...	...
10025	5.0	16.0
10026	2.0	11.0
10027	0.0	0.0
10028	0.0	0.0
10029	0.0	0.0

	vehicles_BUB_IN_0-100_CO2	vehicles_BUB_IN_nan_CO2 \
0	0.0	0.0
1	0.0	0.0
2	0.0	0.0
3	0.0	0.0
4	0.0	0.0
...	...	...
10025	3.0	23.0
10026	3.0	11.0
10027	0.0	0.0
10028	0.0	0.0

10029	0.0	0.0
	vehicles_BUB_IN_201-300_CO2	vehicles_BUB_IN_+300_CO2
0	0.0	0.0
1	0.0	0.0
2	0.0	0.0
3	0.0	0.0
4	0.0	0.0
...	...	...
10025	0.0	0.0
10026	1.0	0.0
10027	0.0	0.0
10028	0.0	0.0
10029	0.0	0.0

[10030 rows x 103 columns]

```
[4]: trafico_contamina_interseccion = pd.read_csv('../data/
      ↪trafico_contamina_intersección.csv')
trafico_contamina_interseccion["Date"] = pd.
      ↪to_datetime(trafico_contamina_interseccion["Date"])
trafico_contamina_interseccion["Date"] = trafico_contamina_interseccion["Date"].
      ↪dt.tz_localize("UTC")
trafico_contamina_interseccion.rename(columns={'Date': 'date'}, inplace=True)
```

```
[5]: columns_interseccion = [
      "CO",
      "NO2",
      "NO",
      "O3",
      "PM10",
      "eBC_ff",
      "eBC_bb",
      "TEMP",
      "RH",
      "WS",
      "WD",
      "PRES"
    ]
# Combinamos ambas listas
columns_interseccion = columns + columns_interseccion

trafico_pam =
      ↪trafico_contamina_interseccion[trafico_contamina_interseccion["truck_pos"]
      ↪== "PAM_2"]

df_int = trafico_pam[["date"] + columns_interseccion]
```

```

columnas_int64 = df_int.select_dtypes(include='int64').columns

# Convertir esas columnas a float64
df_int[columnas_int64] = df_int[columnas_int64].astype('float64')

df_int

```

```

[5]:
      date  vehicles_PAM_1_OUT \
842 2023-01-17 17:00:00+00:00    6.0
843 2023-01-17 18:00:00+00:00    0.0
844 2023-01-17 19:00:00+00:00    0.0
845 2023-01-17 20:00:00+00:00    0.0
846 2023-01-17 21:00:00+00:00    0.0
...
2172 2023-06-26 19:00:00+00:00    7.0
2173 2023-06-26 20:00:00+00:00    3.0
2174 2023-06-26 21:00:00+00:00    0.0
2175 2023-06-26 22:00:00+00:00    0.0
2176 2023-06-27 00:00:00+00:00    0.0

      vehicles_PAM_1_OUT_Zona_Granada \
842                                4.0
843                                0.0
844                                0.0
845                                0.0
846                                0.0
...
2172                                5.0
2173                                3.0
2174                                0.0
2175                                0.0
2176                                0.0

      vehicles_PAM_1_OUT_Zona_Catalunia_y_Otras \
842                                0.0
843                                0.0
844                                0.0
845                                0.0
846                                0.0
...
2172                                1.0
2173                                0.0
2174                                0.0
2175                                0.0
2176                                0.0

      vehicles_PAM_1_OUT_Zona_Andalucia_no_GR  vehicles_PAM_1_OUT_Extranjero \

```


842	0.0	1.0
843	0.0	0.0
844	0.0	0.0
845	0.0	0.0
846	0.0	0.0
...	...	...
2172	0.0	1.0
2173	0.0	0.0
2174	0.0	0.0
2175	0.0	0.0
2176	0.0	0.0

	vehicles_PAM_1_OUT_Zona_Comunidad_de_Madrid \
842	0.0
843	0.0
844	0.0
845	0.0
846	0.0
...	...
2172	0.0
2173	0.0
2174	0.0
2175	0.0
2176	0.0

	vehicles_PAM_1_OUT_Zona_Extremadura_y_Otras \
842	1.0
843	0.0
844	0.0
845	0.0
846	0.0
...	...
2172	0.0
2173	0.0
2174	0.0
2175	0.0
2176	0.0

	vehicles_PAM_1_OUT_Zona_Otras	vehicles_PAM_1_OUT_5_seats	...	NO	\
842	0.0	4.0	...	0.8	
843	0.0	0.0	...	0.5	
844	0.0	0.0	...	0.3	
845	0.0	0.0	...	0.2	
846	0.0	0.0	...	0.2	
...	...	...	...	...	
2172	0.0	5.0	...	1.5	
2173	0.0	3.0	...	0.4	

2174	0.0	0.0	...	0.1
2175	0.0	0.0	...	0.1
2176	0.0	0.0	...	0.1

	O3	PM10	eBC_ff	eBC_bb	TEMP	RH	WS	WD	PRES
842	NaN	15.4	NaN	NaN	9.8	47.7	NaN	NaN	878.7
843	NaN	8.8	NaN	NaN	8.6	54.5	NaN	NaN	878.7
844	NaN	15.7	NaN	NaN	8.3	48.0	NaN	NaN	878.5
845	NaN	10.2	NaN	NaN	7.8	50.2	NaN	NaN	879.0
846	NaN	9.9	NaN	NaN	7.4	52.3	NaN	NaN	878.3
...	...	...	...	...	...	...	...	...	...
2172	111.4	7.8	0.6	0.1	31.4	21.5	1.12	246.34	893.3
2173	99.6	23.0	0.3	0.1	28.8	26.0	0.81	7.46	893.6
2174	78.8	11.3	0.4	0.1	25.7	29.5	1.06	129.23	893.9
2175	77.4	1.9	0.2	0.1	24.7	27.9	1.25	133.45	894.3
2176	84.3	14.3	0.1	0.1	24.2	30.3	1.49	123.34	894.5

[1335 rows x 115 columns]

1.1 Interpolación para completar los valores nulos.

1.2 Rellenar del df_int las horas faltantes con valores NaN

En los datos del camión, poner un registro por cada hora. Si no se encuentra, añadir todos NaN.

El método de pandas `date_range` devuelve el rango de datetimes igualmente espaciados (dada una frecuencia) entre un inicio y un final (por defecto ambos inclusive). El tipo de dato que devuelve es `DatetimeIndex`. Dado que disponemos de dos periodos de fechas con datos disponibles (1er periodo: 17/01/2023-14/03/2023 y 2do periodo 06/06/2023-27/06/2023) realizaremos otro rango para el segundo periodo y lo uniremos mediante el método `union` perteneciente a la clase `Index`.

Explicación paso a paso: - `set_index('date')`: Convierte la columna 'date' en el índice del DataFrame `df_int`. - `reindex(full_date_range)` Rellena el DataFrame para que tenga todas las fechas en `full_date_range`, incluso si algunas no estaban en `df_int`. Esto es útil si hay fechas faltantes y quieres asegurar de que el índice sea continuo. `reset_index()`

Devuelve la columna de fecha como una columna normal en lugar de mantenerla como índice.

```
[6]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Supongamos que el DataFrame inicial es df_int con columnas 'date'

# Definir los dos periodos
start_date_1 = pd.to_datetime("2023-01-17 17:00:00+00:00")
end_date_1 = pd.to_datetime("2023-03-14 11:00:00+00:00")

start_date_2 = pd.to_datetime("2023-06-06 13:00:00+00:00")
```

```

end_date_2 = pd.to_datetime("2023-06-27 00:00:00+00:00")

# Crear un rango completo de fechas con frecuencia horaria para ambos períodos
↳ combinados
full_date_range = pd.date_range(start=start_date_1, end=end_date_1, freq='h').
↳ union(
    pd.date_range(start=start_date_2, end=end_date_2, freq='h')
)

# Asegurar que 'date' sea de tipo datetime
df_int['date'] = pd.to_datetime(df_int['date'])

# Reindexar para completar las horas faltantes
df_int = df_int.set_index('date').reindex(full_date_range).reset_index()
df_int = df_int.rename(columns={'index': 'date'})

df_int

```

```

[6]:
      date  vehicles_PAM_1_OUT \
0  2023-01-17 17:00:00+00:00    6.0
1  2023-01-17 18:00:00+00:00    0.0
2  2023-01-17 19:00:00+00:00    0.0
3  2023-01-17 20:00:00+00:00    0.0
4  2023-01-17 21:00:00+00:00    0.0
...
1826 2023-06-26 20:00:00+00:00    3.0
1827 2023-06-26 21:00:00+00:00    0.0
1828 2023-06-26 22:00:00+00:00    0.0
1829 2023-06-26 23:00:00+00:00    NaN
1830 2023-06-27 00:00:00+00:00    0.0

```

```

      vehicles_PAM_1_OUT_Zona_Granada \
0                                4.0
1                                0.0
2                                0.0
3                                0.0
4                                0.0
...
1826                            3.0
1827                            0.0
1828                            0.0
1829                            NaN
1830                            0.0

```

```

      vehicles_PAM_1_OUT_Zona_Catalunia_y_Otras \
0                                0.0
1                                0.0

```

2	0.0
3	0.0
4	0.0
...	...
1826	0.0
1827	0.0
1828	0.0
1829	NaN
1830	0.0

	vehicles_PAM_1_OUT_Zona_Andalucia_no_GR	vehicles_PAM_1_OUT_Extranjero \
0	0.0	1.0
1	0.0	0.0
2	0.0	0.0
3	0.0	0.0
4	0.0	0.0
...	...	...
1826	0.0	0.0
1827	0.0	0.0
1828	0.0	0.0
1829	NaN	NaN
1830	0.0	0.0

	vehicles_PAM_1_OUT_Zona_Comunidad_de_Madrid \
0	0.0
1	0.0
2	0.0
3	0.0
4	0.0
...	...
1826	0.0
1827	0.0
1828	0.0
1829	NaN
1830	0.0

	vehicles_PAM_1_OUT_Zona_Extremadura_y_Otras \
0	1.0
1	0.0
2	0.0
3	0.0
4	0.0
...	...
1826	0.0
1827	0.0
1828	0.0
1829	NaN

1830

0.0

	vehicles_PAM_1_OUT_Zona_Otras	vehicles_PAM_1_OUT_5_seats	...	NO	\
0	0.0	4.0	...	0.8	
1	0.0	0.0	...	0.5	
2	0.0	0.0	...	0.3	
3	0.0	0.0	...	0.2	
4	0.0	0.0	...	0.2	
...	...	...	...	...	
1826	0.0	3.0	...	0.4	
1827	0.0	0.0	...	0.1	
1828	0.0	0.0	...	0.1	
1829	NaN	NaN	...	NaN	
1830	0.0	0.0	...	0.1	

	O3	PM10	eBC_ff	eBC_bb	TEMP	RH	WS	WD	PRES
0	NaN	15.4	NaN	NaN	9.8	47.7	NaN	NaN	878.7
1	NaN	8.8	NaN	NaN	8.6	54.5	NaN	NaN	878.7
2	NaN	15.7	NaN	NaN	8.3	48.0	NaN	NaN	878.5
3	NaN	10.2	NaN	NaN	7.8	50.2	NaN	NaN	879.0
4	NaN	9.9	NaN	NaN	7.4	52.3	NaN	NaN	878.3
...	...	...	...	...	...	...	...	...	...
1826	99.6	23.0	0.3	0.1	28.8	26.0	0.81	7.46	893.6
1827	78.8	11.3	0.4	0.1	25.7	29.5	1.06	129.23	893.9
1828	77.4	1.9	0.2	0.1	24.7	27.9	1.25	133.45	894.3
1829	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
1830	84.3	14.3	0.1	0.1	24.2	30.3	1.49	123.34	894.5

[1831 rows x 115 columns]

1.3 Periodo_1

```
[7]: len(df_int.columns)
```

```
[7]: 115
```

```
[8]: df_int["date"] = pd.to_datetime(df_int["date"])
df_int.set_index("date", inplace=True)
```

1.4 División Por Periodos

```
[9]: import pandas as pd

# Definir los periodos
periodo_1 = df_int.loc["2023-01-17":"2023-03-14"]
periodo_2 = df_int.loc["2023-06-06":"2023-06-27"]
```

1.5 División en training, validation y test

Para dividir el periodo del 17 de enero de 2023 al 14 de marzo de 2023 en entrenamiento, validación y prueba siguiendo un esquema análogo al del Beijing Multi-Site Air-Quality Dataset, vamos a asignar los datos de manera similar a cómo se hizo en ese conjunto.

El esquema original divide los datos en tres partes:

- **Prueba (10%):** 10% de 56 días = 6 días. Esto se redondearía a 6 días.
- **Validación (10%):** Otro 10% de 56 días = 5 días. Esto también sería 5 días.
- **Entrenamiento (80%):** El resto de los días = 44.8 días, lo que sería 45 días.

1.5.1 Duración total

El periodo de tiempo que mencionas va del 17 de enero de 2023 al 14 de marzo de 2023. Primero calculamos cuántos días tiene este periodo.

Desde el 17 de enero de 2023 hasta el 14 de marzo de 2023, **hay 56 días**.

1.6 Resumen:

- **Conjunto de prueba:** 17 enero - 22 enero 2023 (6 días).
- **Conjunto de validación:** 23 enero - 27 enero 2023 (5 días).
- **Conjunto de entrenamiento:** 28 enero - 14 marzo 2023 (45 días).

```
[10]: periodo_1.loc["2023-01-17":"2023-01-22"]
```

```
[10]:
```

	vehicles_PAM_1_OUT \
date	
2023-01-17 17:00:00+00:00	6.0
2023-01-17 18:00:00+00:00	0.0
2023-01-17 19:00:00+00:00	0.0
2023-01-17 20:00:00+00:00	0.0
2023-01-17 21:00:00+00:00	0.0
...	...
2023-01-22 19:00:00+00:00	0.0
2023-01-22 20:00:00+00:00	0.0
2023-01-22 21:00:00+00:00	0.0
2023-01-22 22:00:00+00:00	NaN
2023-01-22 23:00:00+00:00	0.0

	vehicles_PAM_1_OUT_Zona_Granada \
date	
2023-01-17 17:00:00+00:00	4.0
2023-01-17 18:00:00+00:00	0.0
2023-01-17 19:00:00+00:00	0.0
2023-01-17 20:00:00+00:00	0.0
2023-01-17 21:00:00+00:00	0.0
...	...
2023-01-22 19:00:00+00:00	0.0

2023-01-22 20:00:00+00:00	0.0
2023-01-22 21:00:00+00:00	0.0
2023-01-22 22:00:00+00:00	NaN
2023-01-22 23:00:00+00:00	0.0

vehicles_PAM_1_OUT_Zona_Catalunia_y_Otras \	
date	
2023-01-17 17:00:00+00:00	0.0
2023-01-17 18:00:00+00:00	0.0
2023-01-17 19:00:00+00:00	0.0
2023-01-17 20:00:00+00:00	0.0
2023-01-17 21:00:00+00:00	0.0
...	...
2023-01-22 19:00:00+00:00	0.0
2023-01-22 20:00:00+00:00	0.0
2023-01-22 21:00:00+00:00	0.0
2023-01-22 22:00:00+00:00	NaN
2023-01-22 23:00:00+00:00	0.0

vehicles_PAM_1_OUT_Zona_Andalucia_no_GR \	
date	
2023-01-17 17:00:00+00:00	0.0
2023-01-17 18:00:00+00:00	0.0
2023-01-17 19:00:00+00:00	0.0
2023-01-17 20:00:00+00:00	0.0
2023-01-17 21:00:00+00:00	0.0
...	...
2023-01-22 19:00:00+00:00	0.0
2023-01-22 20:00:00+00:00	0.0
2023-01-22 21:00:00+00:00	0.0
2023-01-22 22:00:00+00:00	NaN
2023-01-22 23:00:00+00:00	0.0

vehicles_PAM_1_OUT_Extranjero \	
date	
2023-01-17 17:00:00+00:00	1.0
2023-01-17 18:00:00+00:00	0.0
2023-01-17 19:00:00+00:00	0.0
2023-01-17 20:00:00+00:00	0.0
2023-01-17 21:00:00+00:00	0.0
...	...
2023-01-22 19:00:00+00:00	0.0
2023-01-22 20:00:00+00:00	0.0
2023-01-22 21:00:00+00:00	0.0
2023-01-22 22:00:00+00:00	NaN
2023-01-22 23:00:00+00:00	0.0

vehicles_PAM_1_OUT_Zona_Comunidad_de_Madrid \	
date	
2023-01-17 17:00:00+00:00	0.0
2023-01-17 18:00:00+00:00	0.0
2023-01-17 19:00:00+00:00	0.0
2023-01-17 20:00:00+00:00	0.0
2023-01-17 21:00:00+00:00	0.0
...	...
2023-01-22 19:00:00+00:00	0.0
2023-01-22 20:00:00+00:00	0.0
2023-01-22 21:00:00+00:00	0.0
2023-01-22 22:00:00+00:00	NaN
2023-01-22 23:00:00+00:00	0.0

vehicles_PAM_1_OUT_Zona_Extremadura_y_Otras \	
date	
2023-01-17 17:00:00+00:00	1.0
2023-01-17 18:00:00+00:00	0.0
2023-01-17 19:00:00+00:00	0.0
2023-01-17 20:00:00+00:00	0.0
2023-01-17 21:00:00+00:00	0.0
...	...
2023-01-22 19:00:00+00:00	0.0
2023-01-22 20:00:00+00:00	0.0
2023-01-22 21:00:00+00:00	0.0
2023-01-22 22:00:00+00:00	NaN
2023-01-22 23:00:00+00:00	0.0

vehicles_PAM_1_OUT_Zona_Otras \	
date	
2023-01-17 17:00:00+00:00	0.0
2023-01-17 18:00:00+00:00	0.0
2023-01-17 19:00:00+00:00	0.0
2023-01-17 20:00:00+00:00	0.0
2023-01-17 21:00:00+00:00	0.0
...	...
2023-01-22 19:00:00+00:00	0.0
2023-01-22 20:00:00+00:00	0.0
2023-01-22 21:00:00+00:00	0.0
2023-01-22 22:00:00+00:00	NaN
2023-01-22 23:00:00+00:00	0.0

vehicles_PAM_1_OUT_5_seats \	
date	
2023-01-17 17:00:00+00:00	4.0
2023-01-17 18:00:00+00:00	0.0
2023-01-17 19:00:00+00:00	0.0

2023-01-17 20:00:00+00:00	0.0
2023-01-17 21:00:00+00:00	0.0
...	...
2023-01-22 19:00:00+00:00	0.0
2023-01-22 20:00:00+00:00	0.0
2023-01-22 21:00:00+00:00	0.0
2023-01-22 22:00:00+00:00	NaN
2023-01-22 23:00:00+00:00	0.0

	vehicles_PAM_1_OUT_+5_seats	...	NO	O3	PM10	\
date		...				
2023-01-17 17:00:00+00:00	0.0	...	0.8	NaN	15.4	
2023-01-17 18:00:00+00:00	0.0	...	0.5	NaN	8.8	
2023-01-17 19:00:00+00:00	0.0	...	0.3	NaN	15.7	
2023-01-17 20:00:00+00:00	0.0	...	0.2	NaN	10.2	
2023-01-17 21:00:00+00:00	0.0	...	0.2	NaN	9.9	
...	...	...	...	...		
2023-01-22 19:00:00+00:00	0.0	...	0.2	68.1	10.1	
2023-01-22 20:00:00+00:00	0.0	...	0.6	62.7	8.4	
2023-01-22 21:00:00+00:00	0.0	...	0.2	65.0	3.3	
2023-01-22 22:00:00+00:00	NaN	...	NaN	NaN	NaN	
2023-01-22 23:00:00+00:00	0.0	...	0.2	56.8	10.3	

	eBC_ff	eBC_bb	TEMP	RH	WS	WD	PRES
date							
2023-01-17 17:00:00+00:00	NaN	NaN	9.8	47.7	NaN	NaN	878.7
2023-01-17 18:00:00+00:00	NaN	NaN	8.6	54.5	NaN	NaN	878.7
2023-01-17 19:00:00+00:00	NaN	NaN	8.3	48.0	NaN	NaN	878.5
2023-01-17 20:00:00+00:00	NaN	NaN	7.8	50.2	NaN	NaN	879.0
2023-01-17 21:00:00+00:00	NaN	NaN	7.4	52.3	NaN	NaN	878.3
...	...	...	...	...	...		
2023-01-22 19:00:00+00:00	NaN	NaN	0.9	77.1	1.38	130.7	890.1
2023-01-22 20:00:00+00:00	NaN	NaN	-0.1	80.5	1.56	143.1	891.0
2023-01-22 21:00:00+00:00	NaN	NaN	-0.5	81.8	1.56	140.6	890.0
2023-01-22 22:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2023-01-22 23:00:00+00:00	NaN	NaN	-1.8	75.7	1.43	138.9	890.5

[127 rows x 114 columns]

```
[11]: # Crear el índice con las fechas y horas de 2023-01-17 00:00:00 hasta
      ↪2023-01-17 17:00:00
date_range = pd.date_range(start="2023-01-17 00:00:00", end="2023-01-17 16:00:
      ↪00", freq="H", tz="UTC")

# Crear el índice con las fechas y horas de 2023-01-17 12:00:00 hasta
      ↪2023-01-17 23:00:00
```

```

date_range_end = pd.date_range(start="2023-03-14 12:00:00", end="2023-03-14 23:
↳00:00", freq="H", tz="UTC")

# Definir las columnas de calidad del aire
columns = ["vehicles_PAM_1_OUT", "CO", "NO2", "NO", "O3", "PM10", "eBC_ff",
↳"eBC_bb", "TEMP", "RH", "WS", "WD", "PRES"]

# Crear un DataFrame con el índice y columnas, relleno los valores con NaN
df = pd.DataFrame(np.nan, index=date_range, columns=columns)
df_end = pd.DataFrame(np.nan, index=date_range_end, columns=columns)
periodo_1_df = pd.concat([df, periodo_1, df_end], axis=0) # axis=0 indica
↳concatenar por filas

```

```

/var/folders/gn/9s3465fs47zbr19dkkb4dxwh0000gn/T/ipykernel_74609/3814031928.py:2
: FutureWarning: 'H' is deprecated and will be removed in a future version,
please use 'h' instead.

```

```

date_range = pd.date_range(start="2023-01-17 00:00:00", end="2023-01-17
16:00:00", freq="H", tz="UTC")
/var/folders/gn/9s3465fs47zbr19dkkb4dxwh0000gn/T/ipykernel_74609/3814031928.py:5
: FutureWarning: 'H' is deprecated and will be removed in a future version,
please use 'h' instead.

```

```

date_range_end = pd.date_range(start="2023-03-14 12:00:00", end="2023-03-14
23:00:00", freq="H", tz="UTC")

```

[12]: periodo_1_df

```

[12]:
vehicles_PAM_1_OUT  CO  NO2  NO  O3  PM10  eBC_ff  \
2023-01-17 00:00:00+00:00    NaN NaN  NaN NaN NaN    NaN    NaN
2023-01-17 01:00:00+00:00    NaN NaN  NaN NaN NaN    NaN    NaN
2023-01-17 02:00:00+00:00    NaN NaN  NaN NaN NaN    NaN    NaN
2023-01-17 03:00:00+00:00    NaN NaN  NaN NaN NaN    NaN    NaN
2023-01-17 04:00:00+00:00    NaN NaN  NaN NaN NaN    NaN    NaN
...
2023-03-14 19:00:00+00:00    NaN NaN  NaN NaN NaN    NaN    NaN
2023-03-14 20:00:00+00:00    NaN NaN  NaN NaN NaN    NaN    NaN
2023-03-14 21:00:00+00:00    NaN NaN  NaN NaN NaN    NaN    NaN
2023-03-14 22:00:00+00:00    NaN NaN  NaN NaN NaN    NaN    NaN
2023-03-14 23:00:00+00:00    NaN NaN  NaN NaN NaN    NaN    NaN

eBC_bb  TEMP  RH  ...  vehicles_BUB_IN_Zona_Otras  \
2023-01-17 00:00:00+00:00    NaN    NaN NaN  ...    NaN
2023-01-17 01:00:00+00:00    NaN    NaN NaN  ...    NaN
2023-01-17 02:00:00+00:00    NaN    NaN NaN  ...    NaN
2023-01-17 03:00:00+00:00    NaN    NaN NaN  ...    NaN
2023-01-17 04:00:00+00:00    NaN    NaN NaN  ...    NaN
...
2023-03-14 19:00:00+00:00    NaN    NaN NaN  ...    NaN

```

2023-03-14 20:00:00+00:00	NaN	NaN	NaN	...	NaN
2023-03-14 21:00:00+00:00	NaN	NaN	NaN	...	NaN
2023-03-14 22:00:00+00:00	NaN	NaN	NaN	...	NaN
2023-03-14 23:00:00+00:00	NaN	NaN	NaN	...	NaN

	vehicles_BUB_IN_5_seats	vehicles_BUB_IN_+5_seats	\
2023-01-17 00:00:00+00:00	NaN	NaN	
2023-01-17 01:00:00+00:00	NaN	NaN	
2023-01-17 02:00:00+00:00	NaN	NaN	
2023-01-17 03:00:00+00:00	NaN	NaN	
2023-01-17 04:00:00+00:00	NaN	NaN	
...	...	...	
2023-03-14 19:00:00+00:00	NaN	NaN	
2023-03-14 20:00:00+00:00	NaN	NaN	
2023-03-14 21:00:00+00:00	NaN	NaN	
2023-03-14 22:00:00+00:00	NaN	NaN	
2023-03-14 23:00:00+00:00	NaN	NaN	

	vehicles_BUB_IN_-5_seats	\
2023-01-17 00:00:00+00:00	NaN	
2023-01-17 01:00:00+00:00	NaN	
2023-01-17 02:00:00+00:00	NaN	
2023-01-17 03:00:00+00:00	NaN	
2023-01-17 04:00:00+00:00	NaN	
...	...	
2023-03-14 19:00:00+00:00	NaN	
2023-03-14 20:00:00+00:00	NaN	
2023-03-14 21:00:00+00:00	NaN	
2023-03-14 22:00:00+00:00	NaN	
2023-03-14 23:00:00+00:00	NaN	

	vehicles_BUB_IN_nan_seats	\
2023-01-17 00:00:00+00:00	NaN	
2023-01-17 01:00:00+00:00	NaN	
2023-01-17 02:00:00+00:00	NaN	
2023-01-17 03:00:00+00:00	NaN	
2023-01-17 04:00:00+00:00	NaN	
...	...	
2023-03-14 19:00:00+00:00	NaN	
2023-03-14 20:00:00+00:00	NaN	
2023-03-14 21:00:00+00:00	NaN	
2023-03-14 22:00:00+00:00	NaN	
2023-03-14 23:00:00+00:00	NaN	

	vehicles_BUB_IN_101-200_CO2	\
2023-01-17 00:00:00+00:00	NaN	
2023-01-17 01:00:00+00:00	NaN	

2023-01-17 02:00:00+00:00	NaN
2023-01-17 03:00:00+00:00	NaN
2023-01-17 04:00:00+00:00	NaN
...	...
2023-03-14 19:00:00+00:00	NaN
2023-03-14 20:00:00+00:00	NaN
2023-03-14 21:00:00+00:00	NaN
2023-03-14 22:00:00+00:00	NaN
2023-03-14 23:00:00+00:00	NaN

	vehicles_BUB_IN_0-100_CO2	vehicles_BUB_IN_nan_CO2	\
2023-01-17 00:00:00+00:00	NaN	NaN	
2023-01-17 01:00:00+00:00	NaN	NaN	
2023-01-17 02:00:00+00:00	NaN	NaN	
2023-01-17 03:00:00+00:00	NaN	NaN	
2023-01-17 04:00:00+00:00	NaN	NaN	
...	...	...	
2023-03-14 19:00:00+00:00	NaN	NaN	
2023-03-14 20:00:00+00:00	NaN	NaN	
2023-03-14 21:00:00+00:00	NaN	NaN	
2023-03-14 22:00:00+00:00	NaN	NaN	
2023-03-14 23:00:00+00:00	NaN	NaN	

	vehicles_BUB_IN_201-300_CO2	\
2023-01-17 00:00:00+00:00	NaN	
2023-01-17 01:00:00+00:00	NaN	
2023-01-17 02:00:00+00:00	NaN	
2023-01-17 03:00:00+00:00	NaN	
2023-01-17 04:00:00+00:00	NaN	
...	...	
2023-03-14 19:00:00+00:00	NaN	
2023-03-14 20:00:00+00:00	NaN	
2023-03-14 21:00:00+00:00	NaN	
2023-03-14 22:00:00+00:00	NaN	
2023-03-14 23:00:00+00:00	NaN	

	vehicles_BUB_IN_+300_CO2
2023-01-17 00:00:00+00:00	NaN
2023-01-17 01:00:00+00:00	NaN
2023-01-17 02:00:00+00:00	NaN
2023-01-17 03:00:00+00:00	NaN
2023-01-17 04:00:00+00:00	NaN
...	...
2023-03-14 19:00:00+00:00	NaN
2023-03-14 20:00:00+00:00	NaN
2023-03-14 21:00:00+00:00	NaN
2023-03-14 22:00:00+00:00	NaN

2023-03-14 23:00:00+00:00

NaN

[1368 rows x 114 columns]

```
[13]: periodo_1_df.loc["2023-01-17":"2023-01-22"]
```

```
[13]:
```

	vehicles_PAM_1_OUT	CO	NO2	NO	O3	PM10	\
2023-01-17 00:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	
2023-01-17 01:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	
2023-01-17 02:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	
2023-01-17 03:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	
2023-01-17 04:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	
...	...	...	...	...	...	...	
2023-01-22 19:00:00+00:00	0.0	0.23	5.6	0.2	68.1	10.1	
2023-01-22 20:00:00+00:00	0.0	0.23	6.8	0.6	62.7	8.4	
2023-01-22 21:00:00+00:00	0.0	0.23	4.5	0.2	65.0	3.3	
2023-01-22 22:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	
2023-01-22 23:00:00+00:00	0.0	0.25	5.2	0.2	56.8	10.3	

	eBC_ff	eBC_bb	TEMP	RH	...	\
2023-01-17 00:00:00+00:00	NaN	NaN	NaN	NaN	...	
2023-01-17 01:00:00+00:00	NaN	NaN	NaN	NaN	...	
2023-01-17 02:00:00+00:00	NaN	NaN	NaN	NaN	...	
2023-01-17 03:00:00+00:00	NaN	NaN	NaN	NaN	...	
2023-01-17 04:00:00+00:00	NaN	NaN	NaN	NaN	...	
...	...	...	...	...	...	
2023-01-22 19:00:00+00:00	NaN	NaN	0.9	77.1	...	
2023-01-22 20:00:00+00:00	NaN	NaN	-0.1	80.5	...	
2023-01-22 21:00:00+00:00	NaN	NaN	-0.5	81.8	...	
2023-01-22 22:00:00+00:00	NaN	NaN	NaN	NaN	...	
2023-01-22 23:00:00+00:00	NaN	NaN	-1.8	75.7	...	

	vehicles_BUB_IN_Zona_Otras	\
2023-01-17 00:00:00+00:00	NaN	
2023-01-17 01:00:00+00:00	NaN	
2023-01-17 02:00:00+00:00	NaN	
2023-01-17 03:00:00+00:00	NaN	
2023-01-17 04:00:00+00:00	NaN	
...	...	
2023-01-22 19:00:00+00:00	0.0	
2023-01-22 20:00:00+00:00	0.0	
2023-01-22 21:00:00+00:00	0.0	
2023-01-22 22:00:00+00:00	NaN	
2023-01-22 23:00:00+00:00	0.0	

	vehicles_BUB_IN_5_seats	vehicles_BUB_IN_+5_seats	\
2023-01-17 00:00:00+00:00	NaN	NaN	

2023-01-17 01:00:00+00:00	NaN	NaN
2023-01-17 02:00:00+00:00	NaN	NaN
2023-01-17 03:00:00+00:00	NaN	NaN
2023-01-17 04:00:00+00:00	NaN	NaN
...	...	...
2023-01-22 19:00:00+00:00	0.0	0.0
2023-01-22 20:00:00+00:00	0.0	0.0
2023-01-22 21:00:00+00:00	0.0	0.0
2023-01-22 22:00:00+00:00	NaN	NaN
2023-01-22 23:00:00+00:00	0.0	0.0

	vehicles_BUB_IN_-5_seats \
2023-01-17 00:00:00+00:00	NaN
2023-01-17 01:00:00+00:00	NaN
2023-01-17 02:00:00+00:00	NaN
2023-01-17 03:00:00+00:00	NaN
2023-01-17 04:00:00+00:00	NaN
...	...
2023-01-22 19:00:00+00:00	0.0
2023-01-22 20:00:00+00:00	0.0
2023-01-22 21:00:00+00:00	0.0
2023-01-22 22:00:00+00:00	NaN
2023-01-22 23:00:00+00:00	0.0

	vehicles_BUB_IN_nan_seats \
2023-01-17 00:00:00+00:00	NaN
2023-01-17 01:00:00+00:00	NaN
2023-01-17 02:00:00+00:00	NaN
2023-01-17 03:00:00+00:00	NaN
2023-01-17 04:00:00+00:00	NaN
...	...
2023-01-22 19:00:00+00:00	0.0
2023-01-22 20:00:00+00:00	0.0
2023-01-22 21:00:00+00:00	0.0
2023-01-22 22:00:00+00:00	NaN
2023-01-22 23:00:00+00:00	0.0

	vehicles_BUB_IN_101-200_CO2 \
2023-01-17 00:00:00+00:00	NaN
2023-01-17 01:00:00+00:00	NaN
2023-01-17 02:00:00+00:00	NaN
2023-01-17 03:00:00+00:00	NaN
2023-01-17 04:00:00+00:00	NaN
...	...
2023-01-22 19:00:00+00:00	0.0
2023-01-22 20:00:00+00:00	0.0
2023-01-22 21:00:00+00:00	0.0

```

2023-01-22 22:00:00+00:00      NaN
2023-01-22 23:00:00+00:00      0.0

vehicles_BUB_IN_0-100_CO2  vehicles_BUB_IN_nan_CO2  \
2023-01-17 00:00:00+00:00      NaN      NaN
2023-01-17 01:00:00+00:00      NaN      NaN
2023-01-17 02:00:00+00:00      NaN      NaN
2023-01-17 03:00:00+00:00      NaN      NaN
2023-01-17 04:00:00+00:00      NaN      NaN
...
2023-01-22 19:00:00+00:00      0.0      0.0
2023-01-22 20:00:00+00:00      0.0      0.0
2023-01-22 21:00:00+00:00      0.0      0.0
2023-01-22 22:00:00+00:00      NaN      NaN
2023-01-22 23:00:00+00:00      0.0      0.0

vehicles_BUB_IN_201-300_CO2  \
2023-01-17 00:00:00+00:00      NaN
2023-01-17 01:00:00+00:00      NaN
2023-01-17 02:00:00+00:00      NaN
2023-01-17 03:00:00+00:00      NaN
2023-01-17 04:00:00+00:00      NaN
...
2023-01-22 19:00:00+00:00      0.0
2023-01-22 20:00:00+00:00      0.0
2023-01-22 21:00:00+00:00      0.0
2023-01-22 22:00:00+00:00      NaN
2023-01-22 23:00:00+00:00      0.0

vehicles_BUB_IN_+300_CO2
2023-01-17 00:00:00+00:00      NaN
2023-01-17 01:00:00+00:00      NaN
2023-01-17 02:00:00+00:00      NaN
2023-01-17 03:00:00+00:00      NaN
2023-01-17 04:00:00+00:00      NaN
...
2023-01-22 19:00:00+00:00      0.0
2023-01-22 20:00:00+00:00      0.0
2023-01-22 21:00:00+00:00      0.0
2023-01-22 22:00:00+00:00      NaN
2023-01-22 23:00:00+00:00      0.0

[144 rows x 114 columns]

```

```

[14]: import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler

```

```

from typing import Dict

def sliding_window(data, n_steps):
    X = []
    for i in range(0, len(data), n_steps):
        # Solo agregamos ventanas completas
        if i + n_steps <= len(data):
            X.append(data[i:i + n_steps])

    # Convertimos la lista de ventanas a un array
    return np.array(X)

def create_missingness(data: np.ndarray, rate: float, pattern: str, **kwargs) → np.ndarray:
    """
    Introduce valores faltantes en el conjunto de datos con un patrón
    específico.
    """
    n_samples, n_features, n_steps = data.shape
    missing_data = data.copy()

    if pattern == "point":
        # Introducir valores faltantes de forma aleatoria (punto a punto)
        for i in range(n_samples):
            for j in range(n_features):
                if np.random.rand() < rate:
                    missing_data[i, j, np.random.randint(n_steps)] = np.nan
    elif pattern == "subseq":
        # Introducir valores faltantes en subsecuencias
        for i in range(n_samples):
            for j in range(n_features):
                if np.random.rand() < rate:
                    start_idx = np.random.randint(n_steps)
                    length = np.random.randint(1, n_steps - start_idx)
                    missing_data[i, j, start_idx:start_idx + length] = np.nan
    elif pattern == "block":
        # Introducir valores faltantes en bloques
        for i in range(n_samples):
            for j in range(n_features):
                if np.random.rand() < rate:
                    block_size = np.random.randint(1, n_steps)
                    missing_data[i, j, :block_size] = np.nan
    return missing_data

```

```

[15]: # Seleccionar solo las columnas necesarias para las características
features = columns_interseccion

```



```
[16]: import pandas as pd
from sklearn.preprocessing import StandardScaler

def preprocess_periodo_1_data(
    periodo_1,
    rate: float,
    n_steps: int,
    pattern: str = "point",
    **kwargs,
) -> dict:

    # Asegúrate de que el índice sea de tipo datetime
    if not pd.api.types.is_datetime64_any_dtype(periodo_1.index):
        periodo_1.index = pd.to_datetime(periodo_1.index)

    # Verificar las fechas en el DataFrame
    print("Fechas en periodo_1:", periodo_1.index.min(), "a", periodo_1.index.
↪max())

    # Filtrar los datos para los conjuntos de prueba, validación y entrenamiento
    # Conjunto de prueba: 17 enero - 26 enero 2023
    mask_test = (periodo_1.index >= "2023-01-17") & (periodo_1.index <
↪"2023-01-23")
    test_set = periodo_1[mask_test]

    # Verificar si hay datos en test_set
    print("Datos en conjunto de prueba:", test_set.shape)

    # Conjunto de validación: 27 enero - 5 febrero 2023
    mask_val = (periodo_1.index >= "2023-01-23") & (periodo_1.index <
↪"2023-01-28")
    val_set = periodo_1[mask_val]

    # Verificar si hay datos en val_set
    print("Datos en conjunto de validación:", val_set.shape)

    # Conjunto de entrenamiento: 6 febrero - 14 marzo 2023
    mask_train = (periodo_1.index >= "2023-01-28") & (periodo_1.index <
↪"2023-03-15")
    train_set = periodo_1[mask_train]

    # Verificar si hay datos en train_set
    print("Datos en conjunto de entrenamiento:", train_set.shape)

    # Verificar si los conjuntos de datos no están vacíos antes de proceder
    if test_set.empty or val_set.empty or train_set.empty:
```

```
        raise ValueError("Uno o más conjuntos de datos están vacíos. Revisa las  
↪ fechas y los filtros.")
```

```
train_set = train_set[features]  
val_set = val_set[features]  
test_set = test_set[features]
```

```
# Normalización de datos
```

```
scaler = StandardScaler()  
train_X = scaler.fit_transform(train_set.loc[:, features])  
val_X = scaler.transform(val_set.loc[:, features])  
test_X = scaler.transform(test_set.loc[:, features])
```

```
# Crear ventanas deslizantes
```

```
train_X = sliding_window(train_X, n_steps)  
val_X = sliding_window(val_X, n_steps)  
test_X = sliding_window(test_X, n_steps)
```

```
# Ensamblar el conjunto de datos final
```

```
processed_dataset = {  
    "n_steps": n_steps,  
    "n_features": train_X.shape[-1], # Número de características  
    "scaler": scaler,  
    "train_X": train_X,  
    "val_X": val_X,  
    "test_X": test_X,  
    "train_X_ori": train_X, # Versión original de los datos de  
↪ entrenamiento  
    "val_X_ori": val_X, # Versión original de los datos de validación  
    "test_X_ori": test_X, # Versión original de los datos de prueba  
}
```

```
if rate > 0:
```

```
    # hold out ground truth in the original data for evaluation  
    train_X_ori = train_X  
    val_X_ori = val_X  
    test_X_ori = test_X
```

```
    # mask values in the train set to keep the same with below validation  
↪ and test sets
```

```
    train_X = create_missingness(train_X, rate, pattern, **kwargs)  
    # mask values in the validation set as ground truth  
    val_X = create_missingness(val_X, rate, pattern, **kwargs)  
    # mask values in the test set as ground truth
```

```

test_X = create_missingness(test_X, rate, pattern, **kwargs)

# Revertir el escalado (transformación inversa)
train_X_not_scale = scaler.inverse_transform(train_X.reshape(-1,
↳train_X.shape[-1])) # Ajuste de forma si es necesario
val_X_not_scale = scaler.inverse_transform(val_X.reshape(-1, val_X.
↳shape[-1])) # Ajuste de forma si es necesario
test_X_not_scale = scaler.inverse_transform(test_X.reshape(-1, test_X.
↳shape[-1])) # Ajuste de forma si es necesario

processed_dataset["train_X"] = train_X
processed_dataset["train_X_not_scale"] = train_X_not_scale
processed_dataset["train_X_ori"] = train_X_ori

processed_dataset["val_X"] = val_X
processed_dataset["val_X_not_scale"] = val_X_not_scale
processed_dataset["val_X_ori"] = val_X_ori

processed_dataset["test_X"] = test_X
processed_dataset["test_X_not_scale"] = test_X_not_scale
processed_dataset["test_X_ori"] = test_X_ori
else:
    logger.warning("rate is 0, no missing values are artificially added.")

return processed_dataset

```

```
[17]: periodo_1_df
```

```

[17]:
vehicles_PAM_1_OUT  CO  NO2  NO  O3  PM10  eBC_ff  \
2023-01-17 00:00:00+00:00  NaN NaN  NaN NaN NaN  NaN  NaN
2023-01-17 01:00:00+00:00  NaN NaN  NaN NaN NaN  NaN  NaN
2023-01-17 02:00:00+00:00  NaN NaN  NaN NaN NaN  NaN  NaN
2023-01-17 03:00:00+00:00  NaN NaN  NaN NaN NaN  NaN  NaN
2023-01-17 04:00:00+00:00  NaN NaN  NaN NaN NaN  NaN  NaN
...
2023-03-14 19:00:00+00:00  NaN NaN  NaN NaN NaN  NaN  NaN
2023-03-14 20:00:00+00:00  NaN NaN  NaN NaN NaN  NaN  NaN
2023-03-14 21:00:00+00:00  NaN NaN  NaN NaN NaN  NaN  NaN
2023-03-14 22:00:00+00:00  NaN NaN  NaN NaN NaN  NaN  NaN
2023-03-14 23:00:00+00:00  NaN NaN  NaN NaN NaN  NaN  NaN

eBC_bb  TEMP  RH  ...  vehicles_BUB_IN_Zona_Otras  \
2023-01-17 00:00:00+00:00  NaN  NaN NaN  ...  NaN
2023-01-17 01:00:00+00:00  NaN  NaN NaN  ...  NaN
2023-01-17 02:00:00+00:00  NaN  NaN NaN  ...  NaN

```

2023-01-17 03:00:00+00:00	NaN	NaN	NaN	...	NaN
2023-01-17 04:00:00+00:00	NaN	NaN	NaN	...	NaN
...	...	...	...	...	...
2023-03-14 19:00:00+00:00	NaN	NaN	NaN	...	NaN
2023-03-14 20:00:00+00:00	NaN	NaN	NaN	...	NaN
2023-03-14 21:00:00+00:00	NaN	NaN	NaN	...	NaN
2023-03-14 22:00:00+00:00	NaN	NaN	NaN	...	NaN
2023-03-14 23:00:00+00:00	NaN	NaN	NaN	...	NaN

	vehicles_BUB_IN_5_seats	vehicles_BUB_IN_+5_seats	\
2023-01-17 00:00:00+00:00	NaN	NaN	
2023-01-17 01:00:00+00:00	NaN	NaN	
2023-01-17 02:00:00+00:00	NaN	NaN	
2023-01-17 03:00:00+00:00	NaN	NaN	
2023-01-17 04:00:00+00:00	NaN	NaN	
...	...	...	
2023-03-14 19:00:00+00:00	NaN	NaN	
2023-03-14 20:00:00+00:00	NaN	NaN	
2023-03-14 21:00:00+00:00	NaN	NaN	
2023-03-14 22:00:00+00:00	NaN	NaN	
2023-03-14 23:00:00+00:00	NaN	NaN	

	vehicles_BUB_IN_-5_seats	\
2023-01-17 00:00:00+00:00	NaN	
2023-01-17 01:00:00+00:00	NaN	
2023-01-17 02:00:00+00:00	NaN	
2023-01-17 03:00:00+00:00	NaN	
2023-01-17 04:00:00+00:00	NaN	
...	...	
2023-03-14 19:00:00+00:00	NaN	
2023-03-14 20:00:00+00:00	NaN	
2023-03-14 21:00:00+00:00	NaN	
2023-03-14 22:00:00+00:00	NaN	
2023-03-14 23:00:00+00:00	NaN	

	vehicles_BUB_IN_nan_seats	\
2023-01-17 00:00:00+00:00	NaN	
2023-01-17 01:00:00+00:00	NaN	
2023-01-17 02:00:00+00:00	NaN	
2023-01-17 03:00:00+00:00	NaN	
2023-01-17 04:00:00+00:00	NaN	
...	...	
2023-03-14 19:00:00+00:00	NaN	
2023-03-14 20:00:00+00:00	NaN	
2023-03-14 21:00:00+00:00	NaN	
2023-03-14 22:00:00+00:00	NaN	
2023-03-14 23:00:00+00:00	NaN	

	vehicles_BUB_IN_101-200_CO2 \
2023-01-17 00:00:00+00:00	NaN
2023-01-17 01:00:00+00:00	NaN
2023-01-17 02:00:00+00:00	NaN
2023-01-17 03:00:00+00:00	NaN
2023-01-17 04:00:00+00:00	NaN
...	...
2023-03-14 19:00:00+00:00	NaN
2023-03-14 20:00:00+00:00	NaN
2023-03-14 21:00:00+00:00	NaN
2023-03-14 22:00:00+00:00	NaN
2023-03-14 23:00:00+00:00	NaN

	vehicles_BUB_IN_0-100_CO2	vehicles_BUB_IN_nan_CO2 \
2023-01-17 00:00:00+00:00	NaN	NaN
2023-01-17 01:00:00+00:00	NaN	NaN
2023-01-17 02:00:00+00:00	NaN	NaN
2023-01-17 03:00:00+00:00	NaN	NaN
2023-01-17 04:00:00+00:00	NaN	NaN
...	...	...
2023-03-14 19:00:00+00:00	NaN	NaN
2023-03-14 20:00:00+00:00	NaN	NaN
2023-03-14 21:00:00+00:00	NaN	NaN
2023-03-14 22:00:00+00:00	NaN	NaN
2023-03-14 23:00:00+00:00	NaN	NaN

	vehicles_BUB_IN_201-300_CO2 \
2023-01-17 00:00:00+00:00	NaN
2023-01-17 01:00:00+00:00	NaN
2023-01-17 02:00:00+00:00	NaN
2023-01-17 03:00:00+00:00	NaN
2023-01-17 04:00:00+00:00	NaN
...	...
2023-03-14 19:00:00+00:00	NaN
2023-03-14 20:00:00+00:00	NaN
2023-03-14 21:00:00+00:00	NaN
2023-03-14 22:00:00+00:00	NaN
2023-03-14 23:00:00+00:00	NaN

	vehicles_BUB_IN_+300_CO2
2023-01-17 00:00:00+00:00	NaN
2023-01-17 01:00:00+00:00	NaN
2023-01-17 02:00:00+00:00	NaN
2023-01-17 03:00:00+00:00	NaN
2023-01-17 04:00:00+00:00	NaN
...	...

```

2023-03-14 19:00:00+00:00      NaN
2023-03-14 20:00:00+00:00      NaN
2023-03-14 21:00:00+00:00      NaN
2023-03-14 22:00:00+00:00      NaN
2023-03-14 23:00:00+00:00      NaN

```

[1368 rows x 114 columns]

```
[18]: print(periodo_1_df.columns)
```

```

Index(['vehicles_PAM_1_OUT', 'CO', 'NO2', 'NO', 'O3', 'PM10', 'eBC_ff',
      'eBC_bb', 'TEMP', 'RH',
      ...,
      'vehicles_BUB_IN_Zona_Otras', 'vehicles_BUB_IN_5_seats',
      'vehicles_BUB_IN_+5_seats', 'vehicles_BUB_IN_-5_seats',
      'vehicles_BUB_IN_nan_seats', 'vehicles_BUB_IN_101-200_CO2',
      'vehicles_BUB_IN_0-100_CO2', 'vehicles_BUB_IN_nan_CO2',
      'vehicles_BUB_IN_201-300_CO2', 'vehicles_BUB_IN_+300_CO2'],
      dtype='object', length=114)

```

```

[19]: # Procesar los datos con una tasa de valores faltantes del 10% y usando el
      ↪ patrón 'point'
periodo_1_dataset = preprocess_periodo_1_data(periodo_1_df, n_steps=24, rate=0.
      ↪ 1, pattern='point')

dataset_for_training = {
    "X": periodo_1_dataset['train_X'],
}

dataset_for_validating = {
    "X": periodo_1_dataset['val_X'],
    "X_ori": periodo_1_dataset['val_X_ori'],
}

dataset_for_testing = {
    "X": periodo_1_dataset['test_X'],
    "X_ori": periodo_1_dataset['test_X_ori']
}

```

Fechas en periodo_1: 2023-01-17 00:00:00+00:00 a 2023-03-14 23:00:00+00:00

Datos en conjunto de prueba: (144, 114)

Datos en conjunto de validación: (120, 114)

Datos en conjunto de entrenamiento: (1104, 114)

```
[20]: dataset_for_testing['X']
```

```

[20]: array([[ nan,      nan,      nan, ...,      nan,
              nan,      nan,

```

```

[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
...,
[-0.59429669, -0.62979971, -0.424822 , ...,      nan,
      nan, -1.85026851],
[-0.59429669, -0.62979971, -0.424822 , ...,      nan,
      nan,      nan],
[-0.59429669, -0.62979971, -0.424822 , ...,  3.95396466,
 -1.03045002, -1.91546803]],

[[-0.59429669, -0.62979971, -0.424822 , ...,  3.2865992 ,
 -0.66972603, -1.89916815],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan],
...,
[-0.59429669, -0.62979971, -0.424822 , ...,  2.56361995,
 -0.18274865, -0.69297712],
[-0.59429669, -0.62979971, -0.424822 , ...,  2.43848892,
 1.45403643, -0.57887797],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan]],

[[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan],
...,
[-0.59429669, -0.62979971, -0.424822 , ..., -0.56465566,
 -0.39317098, -0.36697955],
[-0.59429669, -0.62979971, -0.424822 , ..., -0.63417289,
 -0.30599601, -0.36697955],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan]],

[[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan],
...,

```

```

[-0.59429669, -0.62979971, -0.424822 , ..., 0.01928912,
 -0.40519511, -0.18768088],
[-0.59429669, -0.62979971, -0.424822 , ..., 0.03319257,
 -0.59758123, nan],
[ nan, nan, nan, ..., nan,
 nan, nan]],

[[-0.59429669, -0.62979971, -0.424822 , ..., -0.17535914,
 -0.32403221, -0.10618149],
 [-0.59429669, -0.62979971, -0.424822 , ..., 0.03319257,
 -0.51190929, -0.22028064],
 [ nan, nan, nan, ..., nan,
 nan, nan],
 ...,
 [-0.59429669, -0.62979971, -0.424822 , ..., -0.45342808,
 -0.6306476 , -0.30178003],
 [-0.59429669, -0.62979971, -0.424822 , ..., -0.32829705,
 -0.53896359, -0.38327943],
 [-0.59429669, -0.62979971, -0.424822 , ..., -0.23097293,
 -0.35258953, -0.38327943]],

[[-0.59429669, -0.62979971, -0.424822 , ..., 0.00538568,
 -0.6216295 , -0.28548016],
 [ nan, nan, nan, ..., nan,
 nan, nan],
 [ nan, nan, nan, ..., nan,
 nan, nan],
 ...,
 [-0.59429669, -0.62979971, -0.424822 , ..., 0.04709602,
 -0.2774387 , 0.0568173 ],
 [ nan, nan, nan, ..., nan,
 nan, nan],
 [-0.59429669, -0.62979971, -0.424822 , ..., -0.1336488 ,
 -0.30298998, 0.13831669]]], shape=(6, 24, 114))

```

1.7 Imputación baseline: mediante la mediana

```

[21]: import numpy as np

# Copiar el array original
X_imputed_ori = dataset_for_testing['X'].copy()

# Calcular la mediana de cada columna ignorando los NaN
column_medians = np.nanmedian(X_imputed_ori, axis=0)

# Calcular la media de cada fila ignorando los NaN
row_means = np.nanmean(X_imputed_ori, axis=1)

```



```

# Reemplazar las columnas que son completamente NaN con la media de las filas
# Usamos la media de las filas para las columnas que tienen NaN en la mediana
column_medians[np.isnan(column_medians)] = np.nanmean(row_means)

# Evitar impresión completa de grandes arrays
np.set_printoptions(threshold=10)

# Mostrar las medianas sin imprimir el array completo
print(column_medians)

```

```

[[-0.59429669 -0.62979971 -0.424822    ...  0.00538568 -0.6216295
  -0.28548016]
 [-0.59429669 -0.62979971 -0.424822    ...  0.03319257 -0.51190929
  -0.22028064]
 [-0.15662565 -0.15662565 -0.15662565 ... -0.15662565 -0.15662565
  -0.15662565]
 ...
 [-0.59429669 -0.62979971 -0.424822    ...  0.01928912 -0.39317098
  -0.33437979]
 [-0.59429669 -0.62979971 -0.424822    ... -0.14755224 -0.4224798
  -0.38327943]
 [-0.59429669 -0.62979971 -0.424822    ... -0.1336488  -0.35258953
  -0.38327943]]

```

/var/folders/gn/9s3465fs47zbr19dkkb4dxwh0000gn/T/ipykernel_74609/506266450.py:7:

RuntimeWarning: All-NaN slice encountered

```
column_medians = np.nanmedian(X_imputed_ori, axis=0)
```

/var/folders/gn/9s3465fs47zbr19dkkb4dxwh0000gn/T/ipykernel_74609/506266450.py:10:

RuntimeWarning: Mean of empty slice

```
row_means = np.nanmean(X_imputed_ori, axis=1)
```

```

[22]: # Reemplazar los NaN en cada columna con su respectiva mediana
X_imputed = np.where(np.isnan(X_imputed_ori), column_medians, X_imputed_ori)

# Cambiar la configuración de numpy para mostrar el array completo
np.set_printoptions(threshold=np.inf) # Mostrar todos los elementos

# Limitar la impresión a los primeros 100 elementos, por ejemplo
np.set_printoptions(threshold=100)

# Imprimir las medianas calculadas
X_imputed

```

```

[22]: array([[[-0.59429669, -0.62979971, -0.424822    , ...,  0.00538568,
              -0.6216295 , -0.28548016],
              [-0.59429669, -0.62979971, -0.424822    , ...,  0.03319257,
              -0.51190929, -0.22028064],

```

```

[-0.15662565, -0.15662565, -0.15662565, ..., -0.15662565,
 -0.15662565, -0.15662565],
...,
[-0.59429669, -0.62979971, -0.424822 , ..., 0.01928912,
 -0.39317098, -1.85026851],
[-0.59429669, -0.62979971, -0.424822 , ..., -0.14755224,
 -0.4224798 , -0.38327943],
[-0.59429669, -0.62979971, -0.424822 , ..., 3.95396466,
 -1.03045002, -1.91546803]],

[[-0.59429669, -0.62979971, -0.424822 , ..., 3.2865992 ,
 -0.66972603, -1.89916815],
 [-0.59429669, -0.62979971, -0.424822 , ..., 0.03319257,
 -0.51190929, -0.22028064],
 [-0.15662565, -0.15662565, -0.15662565, ..., -0.15662565,
 -0.15662565, -0.15662565],
 ...,
 [-0.59429669, -0.62979971, -0.424822 , ..., 2.56361995,
 -0.18274865, -0.69297712],
 [-0.59429669, -0.62979971, -0.424822 , ..., 2.43848892,
 1.45403643, -0.57887797],
 [-0.59429669, -0.62979971, -0.424822 , ..., -0.1336488 ,
 -0.35258953, -0.38327943]],

[[-0.59429669, -0.62979971, -0.424822 , ..., 0.00538568,
 -0.6216295 , -0.28548016],
 [-0.59429669, -0.62979971, -0.424822 , ..., 0.03319257,
 -0.51190929, -0.22028064],
 [-0.15662565, -0.15662565, -0.15662565, ..., -0.15662565,
 -0.15662565, -0.15662565],
 ...,
 [-0.59429669, -0.62979971, -0.424822 , ..., -0.56465566,
 -0.39317098, -0.36697955],
 [-0.59429669, -0.62979971, -0.424822 , ..., -0.63417289,
 -0.30599601, -0.36697955],
 [-0.59429669, -0.62979971, -0.424822 , ..., -0.1336488 ,
 -0.35258953, -0.38327943]],

[[-0.59429669, -0.62979971, -0.424822 , ..., 0.00538568,
 -0.6216295 , -0.28548016],
 [-0.59429669, -0.62979971, -0.424822 , ..., 0.03319257,
 -0.51190929, -0.22028064],
 [-0.15662565, -0.15662565, -0.15662565, ..., -0.15662565,
 -0.15662565, -0.15662565],
 ...,
 [-0.59429669, -0.62979971, -0.424822 , ..., 0.01928912,
 -0.40519511, -0.18768088],

```

```

[-0.59429669, -0.62979971, -0.424822 , ..., 0.03319257,
 -0.59758123, -0.38327943],
[-0.59429669, -0.62979971, -0.424822 , ..., -0.1336488 ,
 -0.35258953, -0.38327943]],

[[-0.59429669, -0.62979971, -0.424822 , ..., -0.17535914,
 -0.32403221, -0.10618149],
 [-0.59429669, -0.62979971, -0.424822 , ..., 0.03319257,
 -0.51190929, -0.22028064],
 [-0.15662565, -0.15662565, -0.15662565, ..., -0.15662565,
 -0.15662565, -0.15662565],
 ...,
 [-0.59429669, -0.62979971, -0.424822 , ..., -0.45342808,
 -0.6306476 , -0.30178003],
 [-0.59429669, -0.62979971, -0.424822 , ..., -0.32829705,
 -0.53896359, -0.38327943],
 [-0.59429669, -0.62979971, -0.424822 , ..., -0.23097293,
 -0.35258953, -0.38327943]],

[[-0.59429669, -0.62979971, -0.424822 , ..., 0.00538568,
 -0.6216295 , -0.28548016],
 [-0.59429669, -0.62979971, -0.424822 , ..., 0.03319257,
 -0.51190929, -0.22028064],
 [-0.15662565, -0.15662565, -0.15662565, ..., -0.15662565,
 -0.15662565, -0.15662565],
 ...,
 [-0.59429669, -0.62979971, -0.424822 , ..., 0.04709602,
 -0.2774387 , 0.0568173 ],
 [-0.59429669, -0.62979971, -0.424822 , ..., -0.14755224,
 -0.4224798 , -0.38327943],
 [-0.59429669, -0.62979971, -0.424822 , ..., -0.1336488 ,
 -0.30298998, 0.13831669]]], shape=(6, 24, 114))

```

```
[23]: dataset_for_testing['X']
```

```

[23]: array([[[ nan,      nan,      nan, ...,      nan,
                nan,      nan],
 [ nan,      nan,      nan, ...,      nan,
                nan,      nan],
 [ nan,      nan,      nan, ...,      nan,
                nan,      nan],
 ...,
 [-0.59429669, -0.62979971, -0.424822 , ...,      nan,
                nan, -1.85026851],
 [-0.59429669, -0.62979971, -0.424822 , ...,      nan,
                nan,      nan],
 [-0.59429669, -0.62979971, -0.424822 , ..., 3.95396466,
                nan,      nan]])

```

```

-1.03045002, -1.91546803]],

[[-0.59429669, -0.62979971, -0.424822 , ..., 3.2865992 ,
  -0.66972603, -1.89916815],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan],
 ...,
 [-0.59429669, -0.62979971, -0.424822 , ..., 2.56361995,
  -0.18274865, -0.69297712],
 [-0.59429669, -0.62979971, -0.424822 , ..., 2.43848892,
  1.45403643, -0.57887797],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan]],

[[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan],
 ...,
 [-0.59429669, -0.62979971, -0.424822 , ..., -0.56465566,
  -0.39317098, -0.36697955],
 [-0.59429669, -0.62979971, -0.424822 , ..., -0.63417289,
  -0.30599601, -0.36697955],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan]],

[[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan],
 ...,
 [-0.59429669, -0.62979971, -0.424822 , ..., 0.01928912,
  -0.40519511, -0.18768088],
 [-0.59429669, -0.62979971, -0.424822 , ..., 0.03319257,
  -0.59758123,      nan],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan]],

[[-0.59429669, -0.62979971, -0.424822 , ..., -0.17535914,
  -0.32403221, -0.10618149],
 [-0.59429669, -0.62979971, -0.424822 , ..., 0.03319257,

```

```

-0.51190929, -0.22028064],
[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
...,
[-0.59429669, -0.62979971, -0.424822 , ..., -0.45342808,
 -0.6306476 , -0.30178003],
[-0.59429669, -0.62979971, -0.424822 , ..., -0.32829705,
 -0.53896359, -0.38327943],
[-0.59429669, -0.62979971, -0.424822 , ..., -0.23097293,
 -0.35258953, -0.38327943]],
[[[-0.59429669, -0.62979971, -0.424822 , ..., 0.00538568,
 -0.6216295 , -0.28548016],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan],
 ...,
 [-0.59429669, -0.62979971, -0.424822 , ..., 0.04709602,
 -0.2774387 , 0.0568173 ],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan],
 [-0.59429669, -0.62979971, -0.424822 , ..., -0.1336488 ,
 -0.30298998, 0.13831669]]], shape=(6, 24, 114))

```

1.8 Graficar el primer periodo de las tres variables de contaminación

```

[24]: import matplotlib.pyplot as plt
import warnings

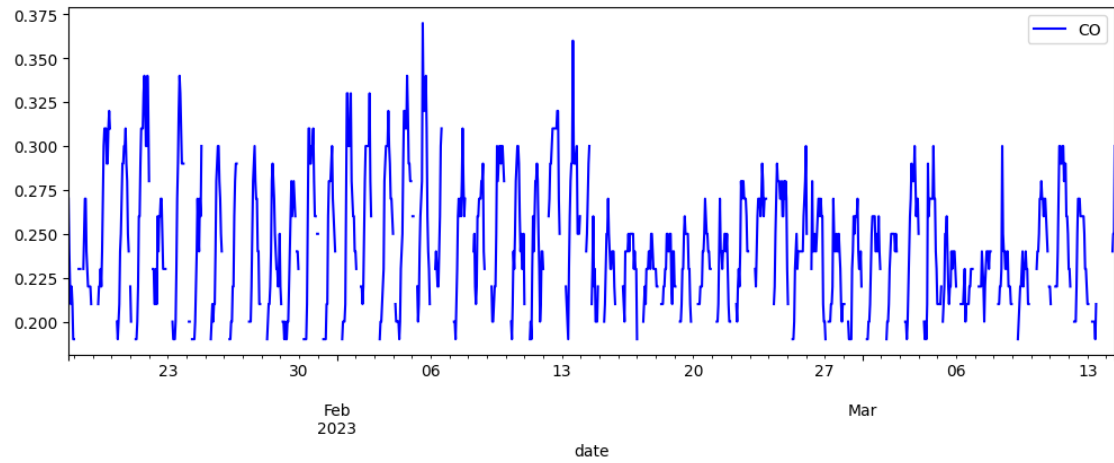
# Configurar el tamaño global de la figura
plt.rcParams["figure.figsize"] = [12,4]

# Graficar con un color personalizado
periodo_1[["CO"]].plot(color='blue')

warnings.filterwarnings("ignore")

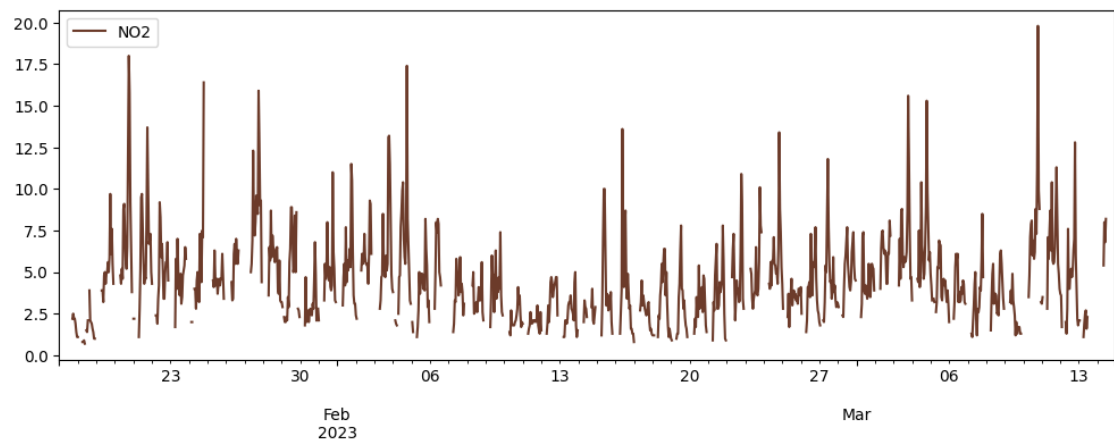
# Mostrar el gráfico
plt.show()

```



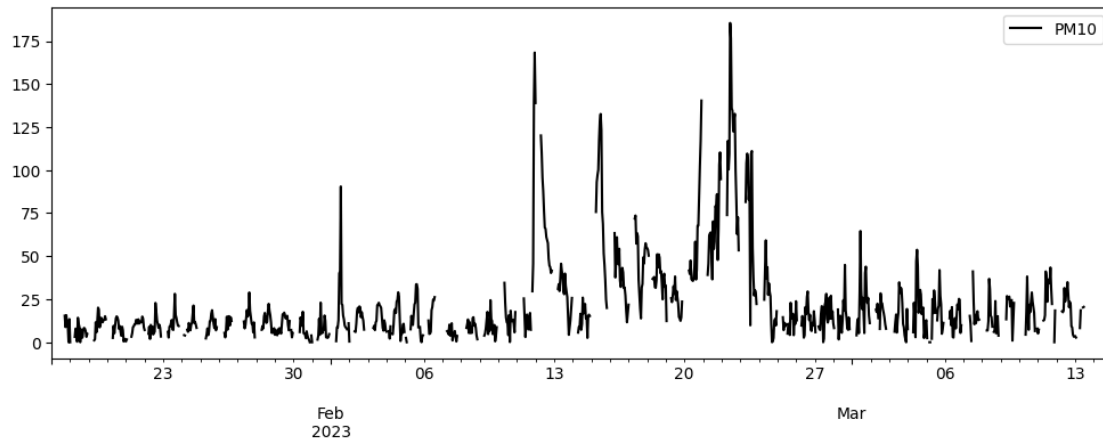
```
[25]: periodo_1_df[["NO2"]].plot(color='#6c3b2a')
```

[25]: <Axes: >



```
[26]: periodo_1_df[["PM10"]].plot(color='black')
```

[26]: <Axes: >



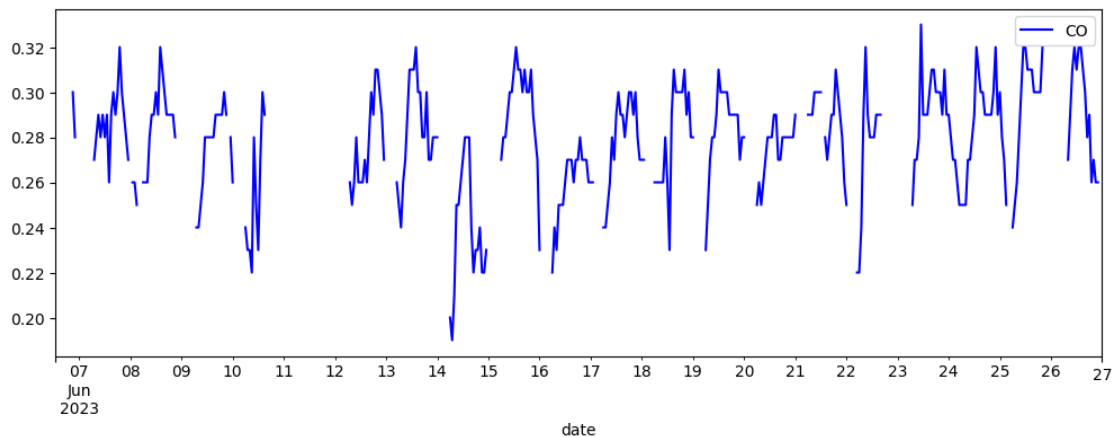
1.9 Graficar el segundo periodo de las tres variables de contaminación

```
[27]: import matplotlib.pyplot as plt

# Configurar el tamaño global de la figura
plt.rcParams["figure.figsize"] = [12,4]

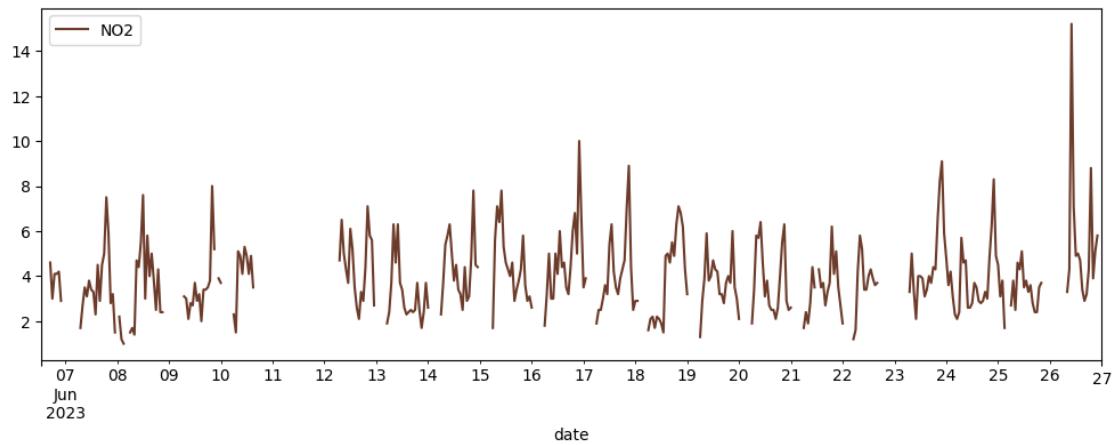
# Graficar con un color personalizado
periodo_2[["CO"]].plot(color='blue')

# Mostrar el gráfico
plt.show()
```



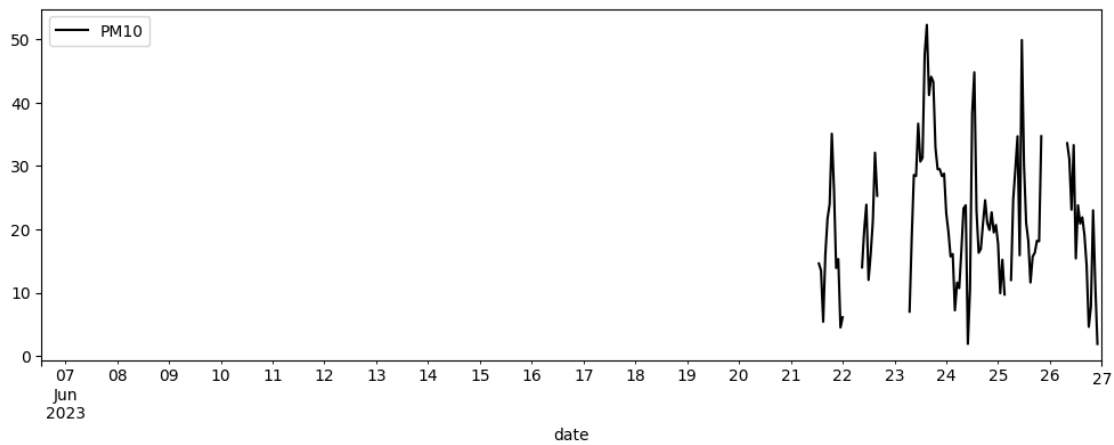
```
[28]: periodo_2[["NO2"]].plot(color='#6c3b2a')
```

```
[28]: <Axes: xlabel='date'>
```



```
[29]: periodo_2[["PM10"]].plot(color='black')
```

```
[29]: <Axes: xlabel='date'>
```



```
[30]: dataset_for_validating
```

[illegible]


```

[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
[      nan,      nan,      nan, ...,      nan,
      nan,      nan]],
[[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan],
...,
[-0.59429669, -0.62979971, -0.424822 , ..., -0.3422005 ,
 -0.69227128, -0.10618149],
[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
[      nan,      nan,      nan, ...,      nan,
      nan,      nan]],
[[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan],
...,
[-0.59429669, -0.62979971, -0.424822 , ...,  0.25564772,
 -0.35860159,  0.25241584],
[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
[-0.59429669, -0.62979971, -0.424822 , ...,  0.28345462,
 -0.47283086,  0.10571693]],
[[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan],
...,
[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan],

```

```

nan, nan]],
[[ nan, nan, nan, ..., nan,
nan, nan],
[ nan, nan, nan, ..., nan,
nan, nan],
[ nan, nan, nan, ..., nan,
nan, nan],
...,
[-0.59429669, -0.62979971, -0.424822 , ..., 0.28345462,
-0.25339043, -1.16567361],
[-0.59429669, -0.62979971, -0.424822 , ..., 0.31126151,
-0.43224941, -1.21457324],
[ nan, nan, nan, ..., nan,
nan, nan]]], shape=(5, 24, 114)),
'X_ori': array([[ nan, nan, nan, ..., nan,
nan, nan],
[ nan, nan, nan, ..., nan,
nan, nan],
[ nan, nan, nan, ..., nan,
nan, nan],
...,
[ nan, nan, nan, ..., nan,
nan, nan],
[ nan, nan, nan, ..., nan,
nan, nan],
[ nan, nan, nan, ..., nan,
nan, nan],
...,
[[ nan, nan, nan, ..., nan,
nan, nan],
[ nan, nan, nan, ..., nan,
nan, nan],
[ nan, nan, nan, ..., nan,
nan, nan],
...,
[-0.59429669, -0.62979971, -0.424822 , ..., -0.3422005 ,
-0.69227128, -0.10618149],
[ nan, nan, nan, ..., nan,
nan, nan],
[ nan, nan, nan, ..., nan,
nan, nan],
[[ nan, nan, nan, ..., nan,
nan, nan],
[ nan, nan, nan, ..., nan,
nan, nan],
[ nan, nan, nan, ..., nan,
nan, nan],
...
```

```

[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
...,
[-0.59429669, -0.62979971, -0.424822 , ...,  0.25564772,
 -0.35860159,  0.25241584],
[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
[-0.59429669, -0.62979971, -0.424822 , ...,  0.28345462,
 -0.47283086,  0.10571693]],

[[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan],
...,
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan]],

[[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan],
...,
 [-0.59429669, -0.62979971, -0.424822 , ...,  0.28345462,
 -0.25339043, -1.16567361],
 [-0.59429669, -0.62979971, -0.424822 , ...,  0.31126151,
 -0.43224941, -1.21457324],
 [      nan,      nan,      nan, ...,      nan,
      nan,      nan]]], shape=(5, 24, 114))}

```

1.10 Imputación usando SAITS

```

[31]: import torch
from torch.nn import SmoothL1Loss
from torch.optim import Adam
from torch.optim.lr_scheduler import ReduceLROnPlateau
from pypots.imputation import SAITS

# Inicializar SAITS con los cambios

```

```

saits = SAITS(
    n_steps=24,
    n_features=periodo_1_dataset['n_features'],
    n_layers=2, # Reducimos capas para evitar sobreajuste
    d_model=128, # Dimensión más pequeña
    d_ffn=64,
    n_heads=4,
    d_k=32,
    d_v=32,
    dropout=0.4, # Mayor dropout para regularizar
    attn_dropout=0.4,
    diagonal_attention_mask=True,
    ORT_weight=1,
    MIT_weight=1,
    batch_size=128,
    epochs=150,
    patience=15,
    num_workers=0,
    device=None,
    saving_path="tutorial_results/imputation/saits",
    model_saving_strategy="best",
)

# Optimizer con menor tasa de aprendizaje y más regularización
optimizer = Adam(saits.model.parameters(), lr=5e-5, weight_decay=1e-4)

# Scheduler que reduce el LR si la pérdida no mejora
scheduler = ReduceLROnPlateau(optimizer, mode="min", factor=0.5, patience=5)

# Función de pérdida SmoothL1Loss en vez de MAE puro
loss_fn = SmoothL1Loss(beta=0.1)

```

```

2025-03-27 16:36:38 [INFO]: No given device, using default device: cpu
2025-03-27 16:36:38 [INFO]: Model files will be saved to
tutorial_results/imputation/saits/20250327_T163638
2025-03-27 16:36:38 [INFO]: Tensorboard file will be saved to
tutorial_results/imputation/saits/20250327_T163638/tensorboard
2025-03-27 16:36:38 [INFO]: Using customized MAE as the training loss function.
2025-03-27 16:36:38 [INFO]: Using customized MSE as the validation metric
function.
2025-03-27 16:36:38 [INFO]: SAITS initialized with the given hyperparameters,
the number of trainable parameters: 447,488

```

ai4ts v0.0.3 - building AI for unified time-series analysis, <https://time-series.ai>

```
[32]: # Entrenar el modelo
      saits.fit(train_set=dataset_for_training, val_set=dataset_for_validating)

2025-03-27 16:36:38 [INFO]: Epoch 001 - training loss (MAE): 1.5261, validation
MSE: 0.6605
2025-03-27 16:36:38 [INFO]: Epoch 002 - training loss (MAE): 1.4480, validation
MSE: 0.4248
2025-03-27 16:36:38 [INFO]: Epoch 003 - training loss (MAE): 1.3889, validation
MSE: 0.2974
2025-03-27 16:36:38 [INFO]: Epoch 004 - training loss (MAE): 1.3463, validation
MSE: 0.2176
2025-03-27 16:36:38 [INFO]: Epoch 005 - training loss (MAE): 1.3056, validation
MSE: 0.1608
2025-03-27 16:36:38 [INFO]: Epoch 006 - training loss (MAE): 1.2756, validation
MSE: 0.1141
2025-03-27 16:36:38 [INFO]: Epoch 007 - training loss (MAE): 1.2313, validation
MSE: 0.0809
2025-03-27 16:36:38 [INFO]: Epoch 008 - training loss (MAE): 1.1822, validation
MSE: 0.0597
2025-03-27 16:36:38 [INFO]: Epoch 009 - training loss (MAE): 1.1321, validation
MSE: 0.0565
2025-03-27 16:36:38 [INFO]: Epoch 010 - training loss (MAE): 1.0971, validation
MSE: 0.0708
2025-03-27 16:36:38 [INFO]: Epoch 011 - training loss (MAE): 1.0658, validation
```

MSE: 0.0790
2025-03-27 16:36:38 [INFO]: Epoch 012 - training loss (MAE): 1.0268, validation
MSE: 0.0684
2025-03-27 16:36:38 [INFO]: Epoch 013 - training loss (MAE): 1.0106, validation
MSE: 0.0450
2025-03-27 16:36:39 [INFO]: Epoch 014 - training loss (MAE): 0.9992, validation
MSE: 0.0298
2025-03-27 16:36:39 [INFO]: Epoch 015 - training loss (MAE): 0.9843, validation
MSE: 0.0242
2025-03-27 16:36:39 [INFO]: Epoch 016 - training loss (MAE): 0.9714, validation
MSE: 0.0227
2025-03-27 16:36:39 [INFO]: Epoch 017 - training loss (MAE): 0.9578, validation
MSE: 0.0231
2025-03-27 16:36:39 [INFO]: Epoch 018 - training loss (MAE): 0.9424, validation
MSE: 0.0205
2025-03-27 16:36:39 [INFO]: Epoch 019 - training loss (MAE): 0.9366, validation
MSE: 0.0158
2025-03-27 16:36:39 [INFO]: Epoch 020 - training loss (MAE): 0.9258, validation
MSE: 0.0103
2025-03-27 16:36:39 [INFO]: Epoch 021 - training loss (MAE): 0.9312, validation
MSE: 0.0056
2025-03-27 16:36:39 [INFO]: Epoch 022 - training loss (MAE): 0.9226, validation
MSE: 0.0032
2025-03-27 16:36:39 [INFO]: Epoch 023 - training loss (MAE): 0.9062, validation
MSE: 0.0021
2025-03-27 16:36:39 [INFO]: Epoch 024 - training loss (MAE): 0.9075, validation
MSE: 0.0012
2025-03-27 16:36:39 [INFO]: Epoch 025 - training loss (MAE): 0.8968, validation
MSE: 0.0010
2025-03-27 16:36:39 [INFO]: Epoch 026 - training loss (MAE): 0.8953, validation
MSE: 0.0023
2025-03-27 16:36:39 [INFO]: Epoch 027 - training loss (MAE): 0.8856, validation
MSE: 0.0047
2025-03-27 16:36:40 [INFO]: Epoch 028 - training loss (MAE): 0.8854, validation
MSE: 0.0074
2025-03-27 16:36:40 [INFO]: Epoch 029 - training loss (MAE): 0.8830, validation
MSE: 0.0086
2025-03-27 16:36:40 [INFO]: Epoch 030 - training loss (MAE): 0.8800, validation
MSE: 0.0088
2025-03-27 16:36:40 [INFO]: Epoch 031 - training loss (MAE): 0.8779, validation
MSE: 0.0080
2025-03-27 16:36:40 [INFO]: Epoch 032 - training loss (MAE): 0.8723, validation
MSE: 0.0078
2025-03-27 16:36:40 [INFO]: Epoch 033 - training loss (MAE): 0.8654, validation
MSE: 0.0076
2025-03-27 16:36:40 [INFO]: Epoch 034 - training loss (MAE): 0.8627, validation
MSE: 0.0066
2025-03-27 16:36:40 [INFO]: Epoch 035 - training loss (MAE): 0.8591, validation

```

MSE: 0.0056
2025-03-27 16:36:40 [INFO]: Epoch 036 - training loss (MAE): 0.8606, validation
MSE: 0.0067
2025-03-27 16:36:40 [INFO]: Epoch 037 - training loss (MAE): 0.8544, validation
MSE: 0.0076
2025-03-27 16:36:40 [INFO]: Epoch 038 - training loss (MAE): 0.8477, validation
MSE: 0.0067
2025-03-27 16:36:40 [INFO]: Epoch 039 - training loss (MAE): 0.8493, validation
MSE: 0.0067
2025-03-27 16:36:40 [INFO]: Epoch 040 - training loss (MAE): 0.8465, validation
MSE: 0.0075
2025-03-27 16:36:40 [INFO]: Exceeded the training patience. Terminating the
training procedure...
2025-03-27 16:36:40 [INFO]: Finished training. The best model is from epoch#25.
2025-03-27 16:36:40 [INFO]: Saved the model to
tutorial_results/imputation/saits/20250327_T163638/SAITS.pypots

```

```

[33]: saits_results = saits.predict(dataset_for_testing)
      saits_imputation = saits_results["imputation"]

```

```

[34]: saits_imputation

```

```

[34]: array([[[-0.49384236, -0.4594484 , -0.57722795, ..., -0.17550409,
               -0.43932745,  0.34693497],
              [-0.492859 , -0.4754197 , -0.5506232 , ..., -0.17644656,
               -0.41046935,  0.37000778],
              [-0.49936843, -0.4768492 , -0.5093403 , ..., -0.16001725,
               -0.39783776,  0.34038824],
              ...,
              [-0.5942967 , -0.6297997 , -0.424822 , ...,  0.02823289,
               -0.5969784 , -1.8502685 ],
              [-0.5942967 , -0.6297997 , -0.424822 , ...,  0.03678316,
               -0.60022706,  0.39938173],
              [-0.5942967 , -0.6297997 , -0.424822 , ...,  3.9539647 ,
               -1.03045 , -1.915468 ]],

              [[-0.5942967 , -0.6297997 , -0.424822 , ...,  3.2865992 ,
               -0.669726 , -1.8991681 ],
              [-0.499125 , -0.47276285, -0.46445978, ..., -0.17613117,
               -0.38840684,  0.33663744],
              [-0.4988696 , -0.477834 , -0.4192621 , ..., -0.15640599,
               -0.3787341 ,  0.29950657],
              ...,
              [-0.5942967 , -0.6297997 , -0.424822 , ...,  2.5636199 ,
               -0.18274865, -0.69297713],
              [-0.5942967 , -0.6297997 , -0.424822 , ...,  2.438489 ,
               1.4540365 , -0.578878 ]],

```

```

[-0.45401633, -0.5616712 , -0.44043764, ..., -0.17067745,
 -0.36288682, 0.44067153]],

[[-0.5358228 , -0.4843598 , -0.524133 , ..., -0.16324323,
 -0.45687753, 0.34229082],
 [-0.5336462 , -0.5061114 , -0.49157944, ..., -0.16889577,
 -0.42937678, 0.36445832],
 [-0.53772634, -0.51241153, -0.44808304, ..., -0.15127647,
 -0.42300493, 0.33245862],
 ...,
 [-0.5942967 , -0.6297997 , -0.424822 , ..., -0.56465566,
 -0.39317098, -0.36697954],
 [-0.5942967 , -0.6297997 , -0.424822 , ..., -0.6341729 ,
 -0.305996 , -0.36697954],
 [-0.48461688, -0.5884061 , -0.459458 , ..., -0.15859555,
 -0.39964017, 0.46473634]],

[[-0.58212125, -0.48096037, -0.51982164, ..., -0.16185266,
 -0.49315268, 0.30474678],
 [-0.57916164, -0.50146675, -0.47891736, ..., -0.16900104,
 -0.4602013 , 0.32241425],
 [-0.5770737 , -0.5075295 , -0.42943895, ..., -0.15202436,
 -0.45198125, 0.2790595 ],
 ...,
 [-0.5942967 , -0.6297997 , -0.424822 , ..., 0.01928912,
 -0.40519512, -0.18768089],
 [-0.5942967 , -0.6297997 , -0.424822 , ..., 0.03319257,
 -0.5975812 , 0.4127682 ],
 [-0.51343924, -0.59591526, -0.45509377, ..., -0.1535503 ,
 -0.4356509 , 0.4541858 ]],

[[-0.5942967 , -0.6297997 , -0.424822 , ..., -0.17535913,
 -0.32403222, -0.10618149],
 [-0.5942967 , -0.6297997 , -0.424822 , ..., 0.03319257,
 -0.5119093 , -0.22028065],
 [-0.5793427 , -0.5013358 , -0.3871555 , ..., -0.1536816 ,
 -0.4289189 , 0.26286155],
 ...,
 [-0.5942967 , -0.6297997 , -0.424822 , ..., -0.4534281 ,
 -0.6306476 , -0.30178005],
 [-0.5942967 , -0.6297997 , -0.424822 , ..., -0.32829705,
 -0.5389636 , -0.38327944],
 [-0.5942967 , -0.6297997 , -0.424822 , ..., -0.23097293,
 -0.35258952, -0.38327944]],

[[-0.5942967 , -0.6297997 , -0.424822 , ..., 0.00538568,
 -0.6216295 , -0.28548014],

```



```

[-0.56517553, -0.47941956, -0.43636835, ..., -0.16720971,
 -0.4209749 ,  0.323592 ],
[-0.55786836, -0.4858612 , -0.38132828, ..., -0.14352082,
 -0.41511664,  0.2756749 ],
...,
[-0.5942967 , -0.6297997 , -0.424822 , ...,  0.04709602,
 -0.2774387 ,  0.0568173 ],
[-0.5192063 , -0.5745944 , -0.4197153 , ..., -0.11655397,
 -0.42878902,  0.44542634],
[-0.5942967 , -0.6297997 , -0.424822 , ..., -0.1336488 ,
 -0.30299 ,  0.13831669]]], shape=(6, 24, 114), dtype=float32)

```

```

[35]: # Crear el índice con las fechas y horas de 2023-01-17 00:00:00 hasta
      ↪ 2023-01-22 23:00:00
date_range = pd.date_range(start="2023-01-17 00:00:00", end="2023-01-22 23:00:
      ↪ 00", freq="h", tz="UTC")

```

```

[36]: import pandas as pd
import numpy as np

# Supongamos que 'saits_imputation' es tu numpy array de shape (6, 24, 23)
# Reestructuramos el array para convertirlo en un formato adecuado para series
      ↪ temporales multivariantes
# Transponemos y aplanamos para que tengamos (6 * 24, 13)

X_reshaped = saits_imputation.reshape(-1, len(features)) # Ahora X_reshaped
      ↪ tiene la forma (6 * 24, 13)

# Crear el índice con las fechas y horas de 2023-01-17 00:00:00 hasta
      ↪ 2023-01-22 23:00:00
date_range = pd.date_range(start="2023-01-17 00:00:00", end="2023-01-22 23:00:
      ↪ 00", freq="h", tz="UTC")

# Creamos el DataFrame
periodo_1_imputed = pd.DataFrame(X_reshaped, columns=features, index=date_range)

# Mostrar las primeras filas del DataFrame
periodo_1_imputed

```

```

[36]:          vehicles_PAM_1_OUT \
2023-01-17 00:00:00+00:00      -0.493842
2023-01-17 01:00:00+00:00      -0.492859
2023-01-17 02:00:00+00:00      -0.499368
2023-01-17 03:00:00+00:00      -0.508321
2023-01-17 04:00:00+00:00      -0.504756

```

...	...
2023-01-22 19:00:00+00:00	-0.594297
2023-01-22 20:00:00+00:00	-0.594297
2023-01-22 21:00:00+00:00	-0.594297
2023-01-22 22:00:00+00:00	-0.519206
2023-01-22 23:00:00+00:00	-0.594297

	vehicles_PAM_1_OUT_Zona_Granada \
2023-01-17 00:00:00+00:00	-0.459448
2023-01-17 01:00:00+00:00	-0.475420
2023-01-17 02:00:00+00:00	-0.476849
2023-01-17 03:00:00+00:00	-0.462469
2023-01-17 04:00:00+00:00	-0.446603

...	...
2023-01-22 19:00:00+00:00	-0.629800
2023-01-22 20:00:00+00:00	-0.629800
2023-01-22 21:00:00+00:00	-0.629800
2023-01-22 22:00:00+00:00	-0.574594
2023-01-22 23:00:00+00:00	-0.629800

	vehicles_PAM_1_OUT_Zona_Catalunia_y_Otras \
2023-01-17 00:00:00+00:00	-0.577228
2023-01-17 01:00:00+00:00	-0.550623
2023-01-17 02:00:00+00:00	-0.509340
2023-01-17 03:00:00+00:00	-0.493742
2023-01-17 04:00:00+00:00	-0.508194

...	...
2023-01-22 19:00:00+00:00	-0.424822
2023-01-22 20:00:00+00:00	-0.424822
2023-01-22 21:00:00+00:00	-0.424822
2023-01-22 22:00:00+00:00	-0.419715
2023-01-22 23:00:00+00:00	-0.424822

	vehicles_PAM_1_OUT_Zona_Andalucia_no_GR \
2023-01-17 00:00:00+00:00	-0.524213
2023-01-17 01:00:00+00:00	-0.544417
2023-01-17 02:00:00+00:00	-0.517031
2023-01-17 03:00:00+00:00	-0.453911
2023-01-17 04:00:00+00:00	-0.397741

...	...
2023-01-22 19:00:00+00:00	-0.412979
2023-01-22 20:00:00+00:00	-0.412979
2023-01-22 21:00:00+00:00	-0.412979
2023-01-22 22:00:00+00:00	-0.442315
2023-01-22 23:00:00+00:00	-0.412979

vehicles_PAM_1_OUT_Extranjero \

2023-01-17 00:00:00+00:00	-0.474127
2023-01-17 01:00:00+00:00	-0.494682
2023-01-17 02:00:00+00:00	-0.506256
2023-01-17 03:00:00+00:00	-0.517957
2023-01-17 04:00:00+00:00	-0.521483
...	...
2023-01-22 19:00:00+00:00	-0.511224
2023-01-22 20:00:00+00:00	-0.511224
2023-01-22 21:00:00+00:00	-0.511224
2023-01-22 22:00:00+00:00	-0.647870
2023-01-22 23:00:00+00:00	-0.511224

	vehicles_PAM_1_OUT_Zona_Comunidad_de_Madrid \
2023-01-17 00:00:00+00:00	-0.425179
2023-01-17 01:00:00+00:00	-0.434498
2023-01-17 02:00:00+00:00	-0.433264
2023-01-17 03:00:00+00:00	-0.425350
2023-01-17 04:00:00+00:00	-0.407803
...	...
2023-01-22 19:00:00+00:00	-0.446205
2023-01-22 20:00:00+00:00	-0.446205
2023-01-22 21:00:00+00:00	-0.446205
2023-01-22 22:00:00+00:00	-0.463694
2023-01-22 23:00:00+00:00	-0.446205

	vehicles_PAM_1_OUT_Zona_Extremadura_y_Otras \
2023-01-17 00:00:00+00:00	-0.413416
2023-01-17 01:00:00+00:00	-0.420403
2023-01-17 02:00:00+00:00	-0.406316
2023-01-17 03:00:00+00:00	-0.372217
2023-01-17 04:00:00+00:00	-0.327823
...	...
2023-01-22 19:00:00+00:00	-0.430105
2023-01-22 20:00:00+00:00	-0.430105
2023-01-22 21:00:00+00:00	-0.430105
2023-01-22 22:00:00+00:00	-0.405247
2023-01-22 23:00:00+00:00	-0.430105

	vehicles_PAM_1_OUT_Zona_Otras \
2023-01-17 00:00:00+00:00	-0.163326
2023-01-17 01:00:00+00:00	-0.138018
2023-01-17 02:00:00+00:00	-0.123040
2023-01-17 03:00:00+00:00	-0.139173
2023-01-17 04:00:00+00:00	-0.176875
...	...
2023-01-22 19:00:00+00:00	-0.124152
2023-01-22 20:00:00+00:00	-0.124152

2023-01-22 21:00:00+00:00	-0.124152
2023-01-22 22:00:00+00:00	-0.149308
2023-01-22 23:00:00+00:00	-0.124152

	vehicles_PAM_1_OUT_5_seats \
2023-01-17 00:00:00+00:00	-0.483935
2023-01-17 01:00:00+00:00	-0.476269
2023-01-17 02:00:00+00:00	-0.440911
2023-01-17 03:00:00+00:00	-0.406108
2023-01-17 04:00:00+00:00	-0.385832
...	...
2023-01-22 19:00:00+00:00	-0.577334
2023-01-22 20:00:00+00:00	-0.577334
2023-01-22 21:00:00+00:00	-0.577334
2023-01-22 22:00:00+00:00	-0.484038
2023-01-22 23:00:00+00:00	-0.577334

	vehicles_PAM_1_OUT_+5_seats	...	NO \
2023-01-17 00:00:00+00:00	-0.315918	...	-0.433373
2023-01-17 01:00:00+00:00	-0.352200	...	-0.417708
2023-01-17 02:00:00+00:00	-0.381757	...	-0.392036
2023-01-17 03:00:00+00:00	-0.384768	...	-0.385389
2023-01-17 04:00:00+00:00	-0.359736	...	-0.392933
...	...	...	...
2023-01-22 19:00:00+00:00	-0.441040	...	-0.616762
2023-01-22 20:00:00+00:00	-0.441040	...	-0.014823
2023-01-22 21:00:00+00:00	-0.441040	...	-0.616762
2023-01-22 22:00:00+00:00	-0.393705	...	-0.732508
2023-01-22 23:00:00+00:00	-0.441040	...	-0.616762

	O3	PM10	eBC_ff	eBC_bb	TEMP \
2023-01-17 00:00:00+00:00	-0.344625	-0.509322	-0.196230	-0.195982	-0.495792
2023-01-17 01:00:00+00:00	-0.336601	-0.490559	-0.198477	-0.197104	-0.491871
2023-01-17 02:00:00+00:00	-0.331027	-0.447631	-0.220815	-0.252631	-0.457293
2023-01-17 03:00:00+00:00	-0.314089	-0.419915	-0.259113	-0.309138	-0.432086
2023-01-17 04:00:00+00:00	-0.294559	-0.418884	-0.287211	-0.329380	-0.416343
...	...	...	...	...	...
2023-01-22 19:00:00+00:00	-0.698750	-0.543959	-0.556383	-0.182650	-1.407906
2023-01-22 20:00:00+00:00	-1.157658	-0.604072	-0.556045	-0.173141	-1.606873
2023-01-22 21:00:00+00:00	-0.962197	-0.784410	-0.563003	-0.172990	-1.686460
2023-01-22 22:00:00+00:00	-0.072679	-0.466266	-0.417144	-0.100410	-0.506489
2023-01-22 23:00:00+00:00	-1.659058	-0.536887	-0.547218	-0.195155	-1.945117

	RH	WS	WD	PRES
2023-01-17 00:00:00+00:00	0.089356	-0.175504	-0.439327	0.346935
2023-01-17 01:00:00+00:00	0.096827	-0.176447	-0.410469	0.370008
2023-01-17 02:00:00+00:00	0.097701	-0.160017	-0.397838	0.340388

```

2023-01-17 03:00:00+00:00    0.083307 -0.146491 -0.421750    0.296513
2023-01-17 04:00:00+00:00    0.057874 -0.142794 -0.444677    0.272262
...
2023-01-22 19:00:00+00:00    1.294200 -0.203166 -0.426237    0.073117
2023-01-22 20:00:00+00:00    1.517024  0.047096 -0.239863    0.219816
2023-01-22 21:00:00+00:00    1.602221  0.047096 -0.277439    0.056817
2023-01-22 22:00:00+00:00    0.145814 -0.116554 -0.428789    0.445426
2023-01-22 23:00:00+00:00    1.202449 -0.133649 -0.302990    0.138317

```

[144 rows x 114 columns]

```

[37]: import pandas as pd
import numpy as np

# Supongamos que 'saits_imputation' es tu numpy array de shape (6, 24, 12)
# Reestructuramos el array para convertirlo en un formato adecuado para series
↳ temporales multivariantes
# Transponemos y aplanamos para que tengamos (6 * 24, 12)

X_resaped = X_imputed.reshape(-1, len(features)) # Ahora X_resaped tiene la
↳ forma (6 * 24, 12)

# Crear el índice con las fechas y horas de 2023-01-17 00:00:00 hasta
↳ 2023-01-22 23:00:00
date_range = pd.date_range(start="2023-01-17 00:00:00", end="2023-01-22 23:00:
↳ 00", freq="h", tz="UTC")

# Creamos el DataFrame
periodo_1_median_imputed = pd.DataFrame(X_resaped, columns=features,
↳ index=date_range)

# Mostrar las primeras filas del DataFrame
periodo_1_median_imputed

```

```

[37]: vehicles_PAM_1_OUT \
2023-01-17 00:00:00+00:00    -0.594297
2023-01-17 01:00:00+00:00    -0.594297
2023-01-17 02:00:00+00:00    -0.156626
2023-01-17 03:00:00+00:00    -0.156626
2023-01-17 04:00:00+00:00    -0.156626
...
2023-01-22 19:00:00+00:00    -0.594297
2023-01-22 20:00:00+00:00    -0.594297
2023-01-22 21:00:00+00:00    -0.594297
2023-01-22 22:00:00+00:00    -0.594297
2023-01-22 23:00:00+00:00    -0.594297

```

	vehicles_PAM_1_OUT_Zona_Granada \
2023-01-17 00:00:00+00:00	-0.629800
2023-01-17 01:00:00+00:00	-0.629800
2023-01-17 02:00:00+00:00	-0.156626
2023-01-17 03:00:00+00:00	-0.156626
2023-01-17 04:00:00+00:00	-0.156626
...	...
2023-01-22 19:00:00+00:00	-0.629800
2023-01-22 20:00:00+00:00	-0.629800
2023-01-22 21:00:00+00:00	-0.629800
2023-01-22 22:00:00+00:00	-0.629800
2023-01-22 23:00:00+00:00	-0.629800

	vehicles_PAM_1_OUT_Zona_Catalunia_y_Otras \
2023-01-17 00:00:00+00:00	-0.424822
2023-01-17 01:00:00+00:00	-0.424822
2023-01-17 02:00:00+00:00	-0.156626
2023-01-17 03:00:00+00:00	-0.156626
2023-01-17 04:00:00+00:00	-0.156626
...	...
2023-01-22 19:00:00+00:00	-0.424822
2023-01-22 20:00:00+00:00	-0.424822
2023-01-22 21:00:00+00:00	-0.424822
2023-01-22 22:00:00+00:00	-0.424822
2023-01-22 23:00:00+00:00	-0.424822

	vehicles_PAM_1_OUT_Zona_Andalucia_no_GR \
2023-01-17 00:00:00+00:00	-0.412979
2023-01-17 01:00:00+00:00	-0.412979
2023-01-17 02:00:00+00:00	-0.156626
2023-01-17 03:00:00+00:00	-0.156626
2023-01-17 04:00:00+00:00	-0.156626
...	...
2023-01-22 19:00:00+00:00	-0.412979
2023-01-22 20:00:00+00:00	-0.412979
2023-01-22 21:00:00+00:00	-0.412979
2023-01-22 22:00:00+00:00	-0.412979
2023-01-22 23:00:00+00:00	-0.412979

	vehicles_PAM_1_OUT_Extranjero \
2023-01-17 00:00:00+00:00	-0.511224
2023-01-17 01:00:00+00:00	-0.511224
2023-01-17 02:00:00+00:00	-0.156626
2023-01-17 03:00:00+00:00	-0.156626
2023-01-17 04:00:00+00:00	-0.156626
...	...
2023-01-22 19:00:00+00:00	-0.511224

2023-01-22 20:00:00+00:00	-0.511224
2023-01-22 21:00:00+00:00	-0.511224
2023-01-22 22:00:00+00:00	-0.511224
2023-01-22 23:00:00+00:00	-0.511224

	vehicles_PAM_1_OUT_Zona_Comunidad_de_Madrid \
2023-01-17 00:00:00+00:00	-0.446205
2023-01-17 01:00:00+00:00	-0.446205
2023-01-17 02:00:00+00:00	-0.156626
2023-01-17 03:00:00+00:00	-0.156626
2023-01-17 04:00:00+00:00	-0.156626
...	...
2023-01-22 19:00:00+00:00	-0.446205
2023-01-22 20:00:00+00:00	-0.446205
2023-01-22 21:00:00+00:00	-0.446205
2023-01-22 22:00:00+00:00	-0.446205
2023-01-22 23:00:00+00:00	-0.446205

	vehicles_PAM_1_OUT_Zona_Extremadura_y_Otras \
2023-01-17 00:00:00+00:00	-0.430105
2023-01-17 01:00:00+00:00	-0.430105
2023-01-17 02:00:00+00:00	-0.156626
2023-01-17 03:00:00+00:00	-0.156626
2023-01-17 04:00:00+00:00	-0.156626
...	...
2023-01-22 19:00:00+00:00	-0.430105
2023-01-22 20:00:00+00:00	-0.430105
2023-01-22 21:00:00+00:00	-0.430105
2023-01-22 22:00:00+00:00	-0.430105
2023-01-22 23:00:00+00:00	-0.430105

	vehicles_PAM_1_OUT_Zona_Otras \
2023-01-17 00:00:00+00:00	-0.124152
2023-01-17 01:00:00+00:00	-0.124152
2023-01-17 02:00:00+00:00	-0.156626
2023-01-17 03:00:00+00:00	-0.156626
2023-01-17 04:00:00+00:00	-0.156626
...	...
2023-01-22 19:00:00+00:00	-0.124152
2023-01-22 20:00:00+00:00	-0.124152
2023-01-22 21:00:00+00:00	-0.124152
2023-01-22 22:00:00+00:00	-0.124152
2023-01-22 23:00:00+00:00	-0.124152

	vehicles_PAM_1_OUT_5_seats \
2023-01-17 00:00:00+00:00	-0.577333
2023-01-17 01:00:00+00:00	-0.577333

2023-01-17 02:00:00+00:00	-0.156626
2023-01-17 03:00:00+00:00	-0.156626
2023-01-17 04:00:00+00:00	-0.156626
...	...
2023-01-22 19:00:00+00:00	-0.577333
2023-01-22 20:00:00+00:00	-0.577333
2023-01-22 21:00:00+00:00	-0.577333
2023-01-22 22:00:00+00:00	-0.577333
2023-01-22 23:00:00+00:00	-0.577333

	vehicles_PAM_1_OUT_+5_seats	...	NO	\
2023-01-17 00:00:00+00:00	-0.441040	...	-0.616762	
2023-01-17 01:00:00+00:00	-0.441040	...	-0.466277	
2023-01-17 02:00:00+00:00	-0.156626	...	-0.156626	
2023-01-17 03:00:00+00:00	-0.156626	...	-0.156626	
2023-01-17 04:00:00+00:00	-0.156626	...	-0.156626	
...	...	...	...	
2023-01-22 19:00:00+00:00	-0.441040	...	-0.616762	
2023-01-22 20:00:00+00:00	-0.441040	...	-0.014823	
2023-01-22 21:00:00+00:00	-0.441040	...	-0.616762	
2023-01-22 22:00:00+00:00	-0.441040	...	-0.466277	
2023-01-22 23:00:00+00:00	-0.441040	...	-0.616762	

	O3	PM10	eBC_ff	eBC_bb	TEMP	\
2023-01-17 00:00:00+00:00	-2.432403	-0.879883	-0.156626	-0.156626	-0.631935	
2023-01-17 01:00:00+00:00	-1.625065	-0.844523	-0.156626	-0.156626	-0.711522	
2023-01-17 02:00:00+00:00	-0.156626	-0.156626	-0.156626	-0.156626	-0.156626	
2023-01-17 03:00:00+00:00	-0.156626	-0.156626	-0.156626	-0.156626	-0.156626	
2023-01-17 04:00:00+00:00	-0.156626	-0.156626	-0.156626	-0.156626	-0.156626	
...	...	...	...	...	...	
2023-01-22 19:00:00+00:00	-0.698750	-0.543959	-0.156626	-0.156626	-1.407906	
2023-01-22 20:00:00+00:00	-1.157658	-0.604072	-0.156626	-0.156626	-1.606873	
2023-01-22 21:00:00+00:00	-0.962197	-0.784410	-0.156626	-0.156626	-1.686460	
2023-01-22 22:00:00+00:00	-1.765287	-0.731369	-0.156626	-0.156626	-0.791109	
2023-01-22 23:00:00+00:00	-1.659058	-0.536887	-0.156626	-0.156626	-1.945116	

	RH	WS	WD	PRES
2023-01-17 00:00:00+00:00	1.523577	0.005386	-0.621630	-0.285480
2023-01-17 01:00:00+00:00	1.523577	0.033193	-0.511909	-0.220281
2023-01-17 02:00:00+00:00	-0.156626	-0.156626	-0.156626	-0.156626
2023-01-17 03:00:00+00:00	-0.156626	-0.156626	-0.156626	-0.156626
2023-01-17 04:00:00+00:00	-0.156626	-0.156626	-0.156626	-0.156626
...	...	...	...	...
2023-01-22 19:00:00+00:00	1.294200	-0.203166	-0.426237	0.073117
2023-01-22 20:00:00+00:00	1.517024	0.047096	-0.239863	0.219816
2023-01-22 21:00:00+00:00	1.602221	0.047096	-0.277439	0.056817
2023-01-22 22:00:00+00:00	1.507193	-0.147552	-0.422480	-0.383279


```
2023-01-22 23:00:00+00:00 1.202449 -0.133649 -0.302990 0.138317
```

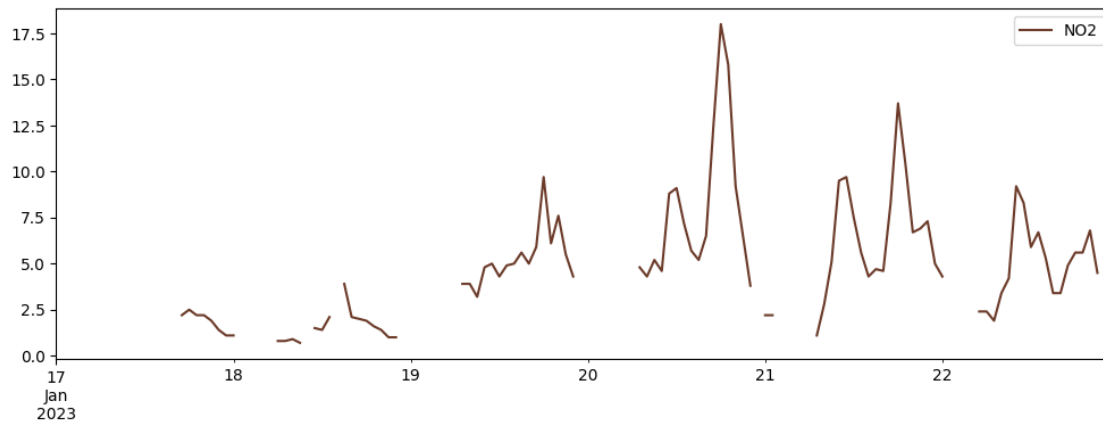
```
[144 rows x 114 columns]
```

1.11 Comparación de gráficas con los datos imputados

1.12 Serie Temporal NO2 con Datos Faltantes

```
[38]: periodo_1_df.loc["2023-01-17":"2023-01-22"][["NO2"]].plot(color='#6c3b2a')
```

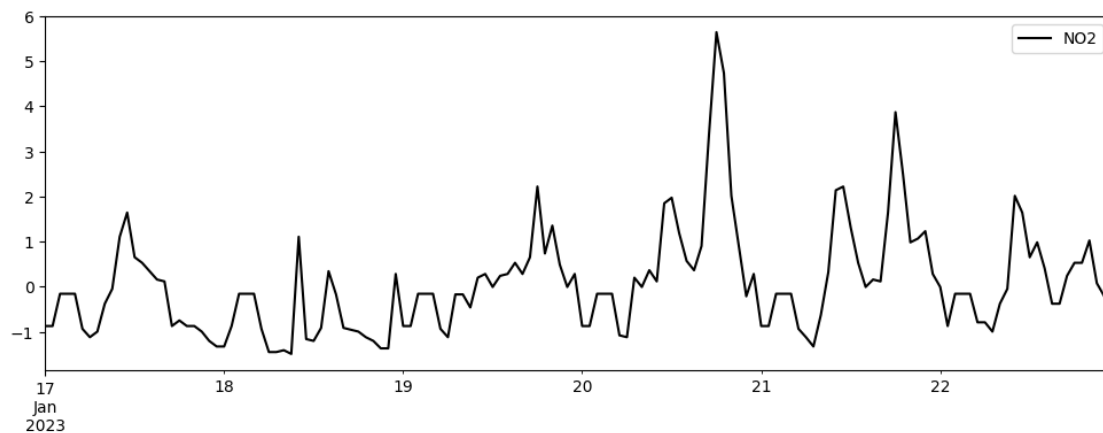
```
[38]: <Axes: >
```



1.13 Serie Temporal NO2 imputada con la mediana

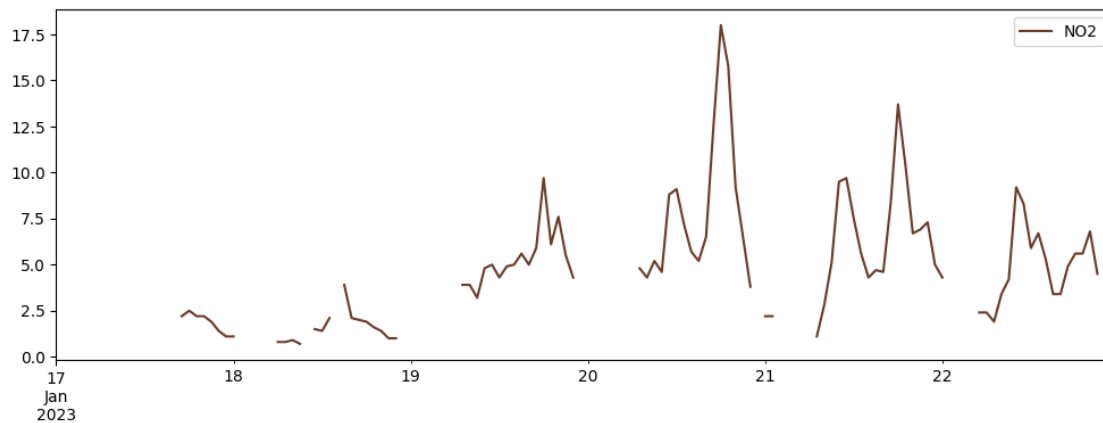
```
[39]: periodo_1_median_imputed.loc["2023-01-17":"2023-01-22"][["NO2"]].  
      ↪plot(color='black')
```

```
[39]: <Axes: >
```



```
[40]: periodo_1_df.loc["2023-01-17":"2023-01-22"][["NO2"]].plot(color='#6c3b2a')
```

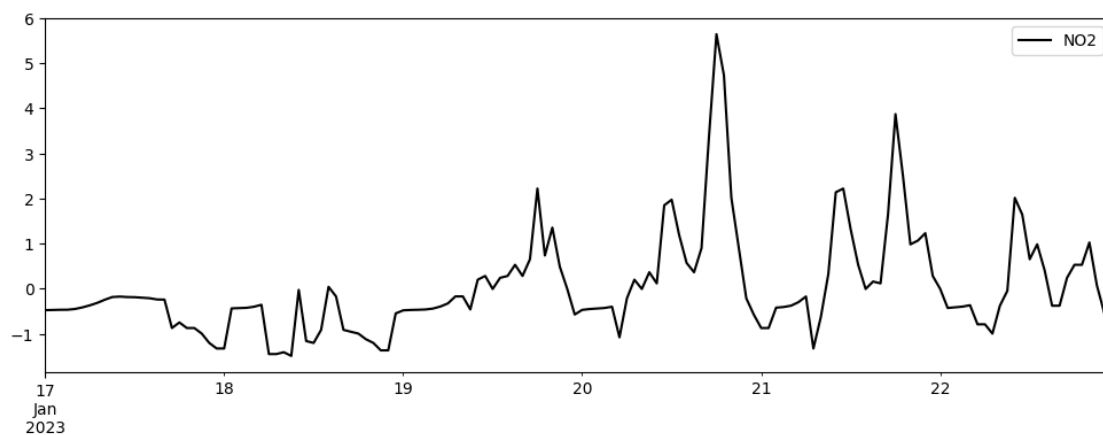
```
[40]: <Axes: >
```



1.14 Serie Temporal NO2 imputada mediante SAITS

```
[41]: periodo_1_imputed.loc["2023-01-17":"2023-01-22"][["NO2"]].plot(color='black')
```

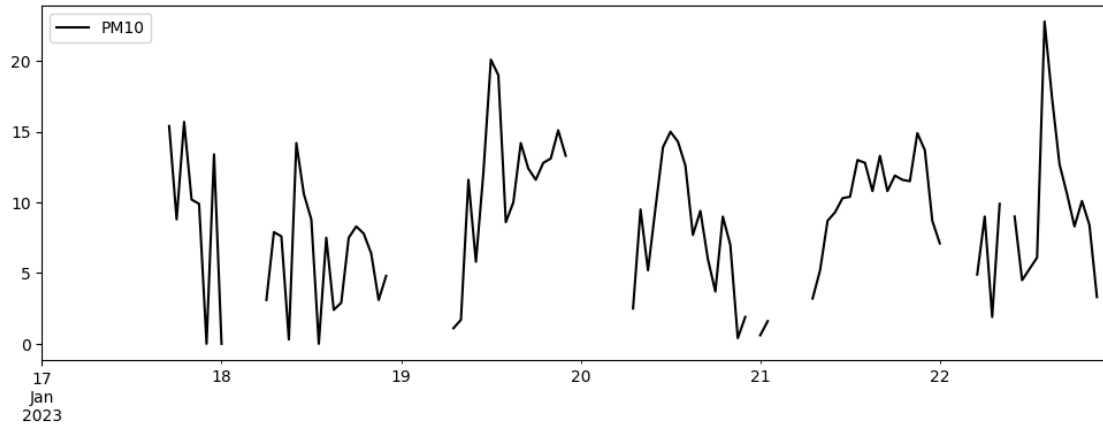
```
[41]: <Axes: >
```



1.15 Serie PM10 con datos faltantes

```
[42]: periodo_1_df.loc["2023-01-17":"2023-01-22"][["PM10"]].plot(color='black')
```

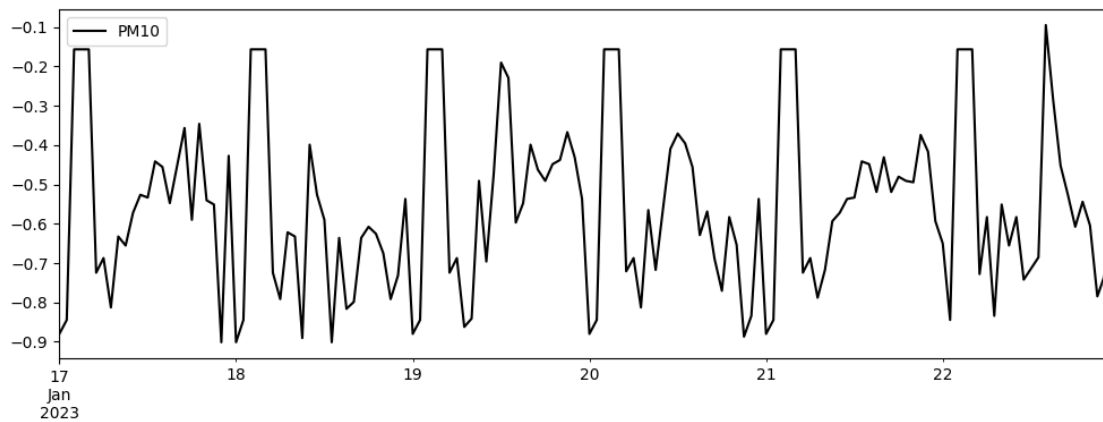
[42]: <Axes: >



1.16 Serie PM10 imputada mediante la mediana

```
[43]: periodo_1_median_imputed.loc["2023-01-17":"2023-01-22"][["PM10"]].  
      plot(color='black')
```

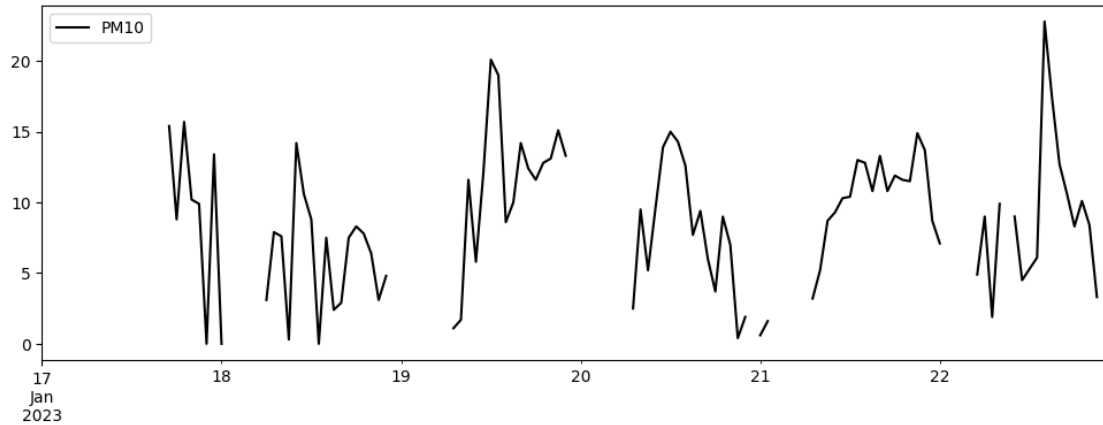
[43]: <Axes: >



1.17 Serie PM10 con datos faltantes

```
[44]: periodo_1_df.loc["2023-01-17":"2023-01-22"][["PM10"]].plot(color='black')
```

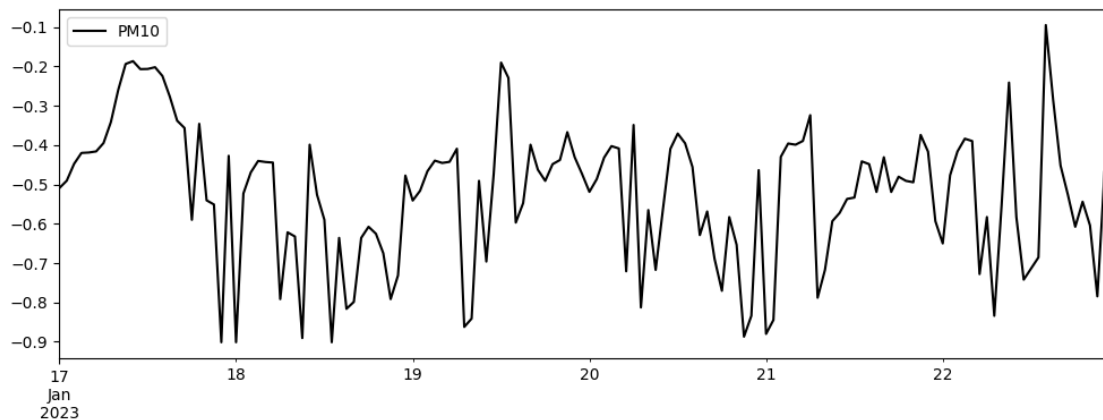
```
[44]: <Axes: >
```



1.18 Serie PM10 imputada mediante SAITS

```
[45]: periodo_1_imputed.loc["2023-01-17":"2023-01-22"][["PM10"]].plot(color='black')
```

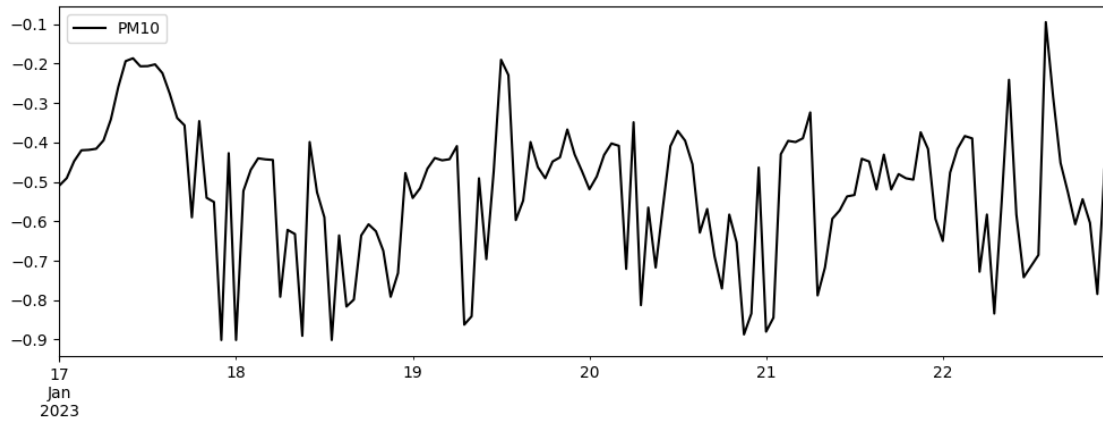
```
[45]: <Axes: >
```



1.19 Serie Temporal NO2 imputada mediante SAITS

```
[46]: periodo_1_imputed.loc["2023-01-17":"2023-01-22"][["PM10"]].plot(color='black')
```

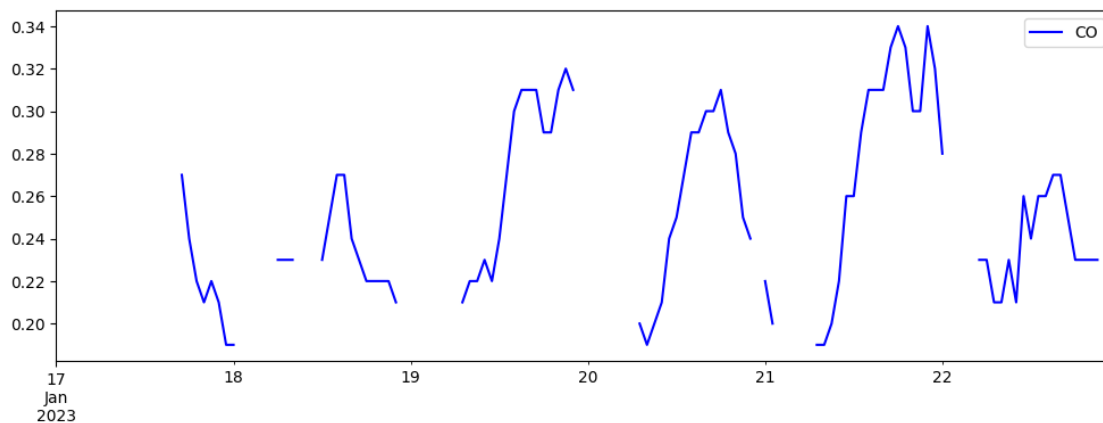
```
[46]: <Axes: >
```



1.20 Serie Temporal CO datos faltantes

```
[47]: periodo_1_df.loc["2023-01-17":"2023-01-22"][["CO"]].plot(color='blue')
```

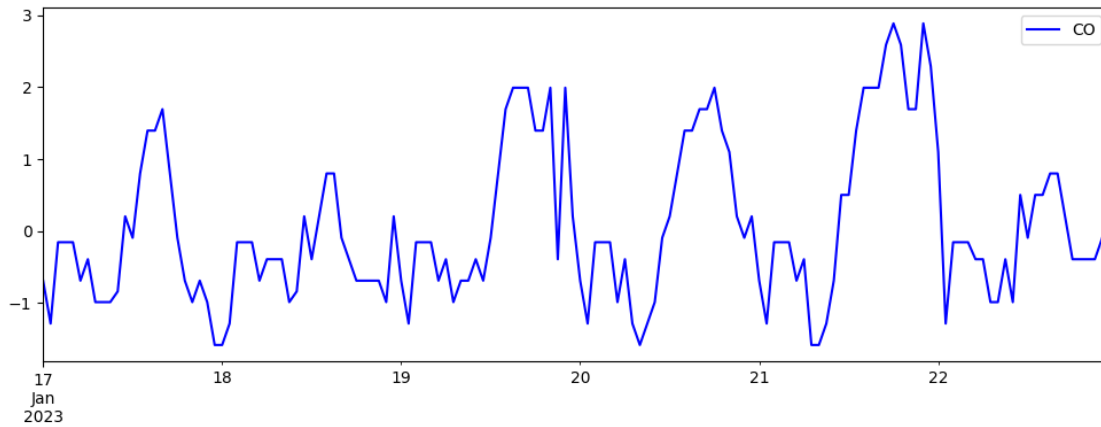
[47]: <Axes: >



1.21 Serie Temporal CO imputada mediante la mediana

```
[48]: periodo_1_median_imputed.loc["2023-01-17":"2023-01-22"][["CO"]].  
      ↪plot(color='blue')
```

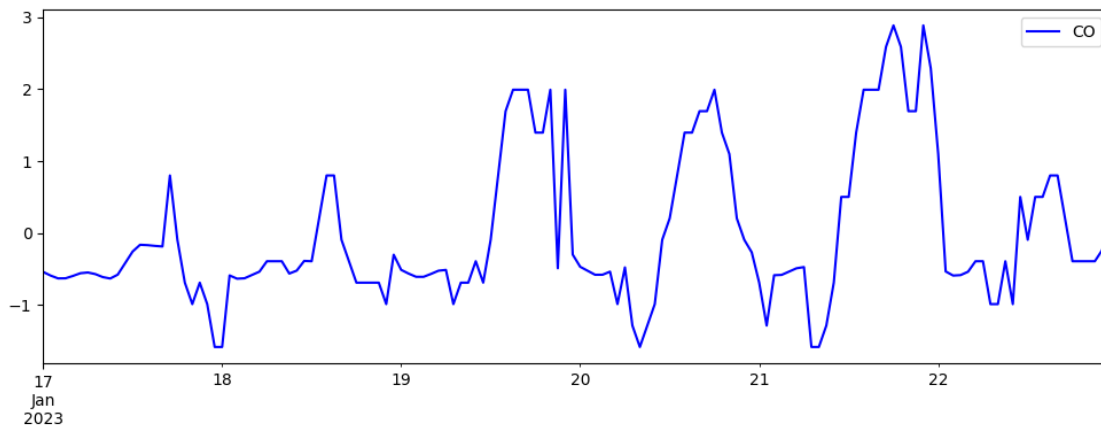
[48]: <Axes: >



1.22 Serie Temporal CO imputada mediante SAITS

```
[49]: periodo_1_imputed.loc["2023-01-17":"2023-01-22"][["CO"]].plot(color='blue')
```

[49]: <Axes: >



2 Comparación de SAITS con métodos baseline

2.1 Imputación con la mediana

```
[50]: from pypots.nn.functional import calc_mae, calc_mse, calc_rmse, calc_mre

periodo_1_dataset['X Indicating Mask'] = np.isnan(dataset_for_testing['X']).
    astype(int)

# Crear una copia de los datos originales
```

```

X_ori_fixed = np.copy(periodo_1_dataset['test_X_ori'])

# Reemplazar NaNs por 0 (valor constante)
X_ori_fixed[np.isnan(X_ori_fixed)] = 0

# Verificar si quedan NaNs
num_nans_fixed = np.isnan(X_ori_fixed).sum()
print(f"Número de NaNs en test_X_ori después de imputación (con 0):  

↳{num_nans_fixed}")

# Calculate Mean Squared Error (MSE)
mtesting_mae = calc_mae(
    X_imputed,
    X_ori_fixed,
    periodo_1_dataset['X_indicating_mask'],
)

# Calculate Mean Squared Error (MSE)
mtesting_mse = calc_mse(
    X_imputed,
    X_ori_fixed,
    periodo_1_dataset['X_indicating_mask'],
)

# Calculate Root Mean Squared Error (RMSE)
mtesting_rmse = calc_rmse(
    X_imputed,
    X_ori_fixed,
    periodo_1_dataset['X_indicating_mask'],
)

# Calculate Mean Relative Error (MRE)
mtesting_mre = calc_mre(
    X_imputed,
    X_ori_fixed,
    periodo_1_dataset['X_indicating_mask'],
)

# Print the errors
print(f"Testing Mean Absolute Error (MAE): {mtesting_mae:.4f}")
print(f"Testing Mean Squared Error (MSE): {mtesting_mse:.4f}")
print(f"Testing Root Mean Squared Error (RMSE): {mtesting_rmse:.4f}")
print(f"Testing Mean Relative Error (MRE): {mtesting_mre:.4f}")

```

Número de NaNs en test_X_ori después de imputación (con 0): 0
 Testing Mean Absolute Error (MAE): 0.4407
 Testing Mean Squared Error (MSE): 0.3286
 Testing Root Mean Squared Error (RMSE): 0.5732

Testing Mean Relative Error (MRE): 432.6484

2.2 Imputación con SAITS

```
[51]: from pypots.nn.functional import calc_mae, calc_mse, calc_rmse, calc_mre

periodo_1_dataset['X_indicating_mask'] = np.isnan(periodo_1_dataset['test_X']).
    ↪astype(int)

# Crear una copia de los datos originales
X_ori_fixed = np.copy(periodo_1_dataset['test_X_ori'])

# Reemplazar NaNs por 0 (valor constante)
X_ori_fixed[np.isnan(X_ori_fixed)] = 0

# Verificar si quedan NaNs
num_nans_fixed = np.isnan(X_ori_fixed).sum()
print(f"Número de NaNs en test_X_ori después de imputación (con 0):_
    ↪{num_nans_fixed}")

# Calculate Mean Squared Error (MSE)
testing_mae = calc_mae(
    saits_imputation,
    X_ori_fixed,
    periodo_1_dataset['X_indicating_mask'],
)

# Calculate Mean Squared Error (MSE)
testing_mse = calc_mse(
    saits_imputation,
    X_ori_fixed,
    periodo_1_dataset['X_indicating_mask'],
)

# Calculate Root Mean Squared Error (RMSE)
testing_rmse = calc_rmse(
    saits_imputation,
    X_ori_fixed,
    periodo_1_dataset['X_indicating_mask'],
)

# Calculate Mean Relative Error (MRE)
testing_mre = calc_mre(
    saits_imputation,
    X_ori_fixed,
    periodo_1_dataset['X_indicating_mask'],
)
```



```

# Print the errors
print(f"Testing Mean Absolute Error (MAE): {testing_mae:.4f}")
print(f"Testing Mean Squared Error (MSE): {testing_mse:.4f}")
print(f"Testing Root Mean Squared Error (RMSE): {testing_rmse:.4f}")
print(f"Testing Mean Relative Error (MRE): {testing_mre:.4f}")

```

Número de NaNs en test_X_ori después de imputación (con 0): 0
 Testing Mean Absolute Error (MAE): 0.3693
 Testing Mean Squared Error (MSE): 0.1696
 Testing Root Mean Squared Error (RMSE): 0.4119
 Testing Mean Relative Error (MRE): 362.5761

```

[52]: import pandas as pd

# Crear un DataFrame con los errores de ambos métodos, reorganizando para que
# RMSE aparezca primero
error_data = {
    "Error Type": ["Root Mean Squared Error (RMSE)", "Mean Squared Error",
    "(MSE)", "Mean Absolute Error (MAE)",
    "Mean Relative Error (MRE)"],
    "Median Imputation": [mtesting_rmse, mtesting_mse, mtesting_mae,
    mtesting_mre],
    "SAITS Imputation": [testing_rmse, testing_mse, testing_mae, testing_mre]
}

# Convertir el diccionario en un DataFrame
error_table = pd.DataFrame(error_data)

error_table

```

```

[52]:

```

	Error Type	Median Imputation	SAITS Imputation
0	Root Mean Squared Error (RMSE)	0.573211	0.411882
1	Mean Squared Error (MSE)	0.328571	0.169647
2	Mean Absolute Error (MAE)	0.440679	0.369306
3	Mean Relative Error (MRE)	432.648363	362.576086